Deep Clustering via Joint Variational Autoencoder and Non-negative Matrix Factorization with Proximal Gradient Optimization

Hang Lyu

November 8, 2026

Abstract

Clustering, a foundational task in unsupervised machine learning, aims at uncovering inherent patterns within data. Traditional clustering techniques often struggle with high-dimensional datasets due to computational inefficiency, sensitivity to initialization, and the curse of dimensionality, resulting in suboptimal clustering performance. To address these limitations, this study proposes a deep clustering model that integrates Variational Autoencoder (VAE)-based representation learning and Non-negative Matrix Factorization (NMF)-based clustering. The method employs alternating optimization combined with proximal gradient descent to jointly optimize the representation and clustering objectives.

Experimental evaluations were conducted using widely recognized benchmarks, including the MNIST and REUTERS-10k datasets. The results demonstrate significant improvements over traditional clustering algorithms, with consistent performance gains observed in accuracy, normalized mutual information, adjusted Rand index, purity, and F1-measure. Visualizations of the latent space clearly illustrate well-separated clusters, validating the effectiveness of the proposed representation learning. Moreover, the confusion maps provide insights into the clustering dynamics, highlighting areas of ambiguity and potential improvements. Investigations into the optimal number of clusters further confirm that the model effectively balances complexity and interpretability.

This manuscript offers an efficient and scalable deep clustering framework capable of handling high-dimensional data, combining robust feature extraction capabilities of deep generative models with the interpretability and efficiency of matrix factorization methods.

Contents

1 Introduction **4**
- 1.1 Problem Formulation . . . 5
- 1.2 Main Results . . . 5
- 1.3 Contributions . . . 6
- 1.4 Outline . . . 6

2 Background **7**
- 2.1 Traditional Clustering . . . 8
  - 2.1.1 k-means Clustering . . . 8
  - 2.1.2 Non-negative Matrix Factorization . . . 9
  - 2.1.3 Relationship between k-means Clustering and NMF . . . 10
- 2.2 Representation Module . . . 11
  - 2.2.1 Feedforward Network . . . 11
  - 2.2.2 Autoencoder . . . 12
  - 2.2.3 Deep Generative Model: VAE . . . 12
- 2.3 Variational Autoencoder: Detailed derivation and implementation 13
  - 2.3.1 Loss Function . . . 14
  - 2.3.2 Optimization Strategy . . . 15
- 2.4 Binary Constraints . . . 18
  - 2.4.1 Binary Regularization . . . 18
  - 2.4.2 Exclusive Binary Restriction . . . 18

3 Related Work **20**
- 3.1 Distance–minimization Methods . . . 20
- 3.2 Probability Distribution Methods . . . 21
- 3.3 SpectralNet . . . 23

4 Method **24**
- 4.1 Proposed Model . . . 25
- 4.2 Objective Function . . . 25
- 4.3 Training Strategy . . . 26
  - 4.3.1 Alternating Optimization . . . 26
  - 4.3.2 Proximal Gradient Descent . . . 27

5 Experiments **29**
- 5.1 Datasets . . . 29
  - 5.1.1 MNIST . . . 29
  - 5.1.2 REUTERS-10k . . . 29
- 5.2 Evaluation Metrics . . . 29
  - 5.2.1 Accuracy . . . 30
  - 5.2.2 Normalized Mutual Information . . . 30
  - 5.2.3 Adjusted Rand Index . . . 30
  - 5.2.4 Purity . . . 31
  - 5.2.5 F1_measure . . . 31
- 5.3 Implementation . . . 31
  - 5.3.1 Model Architectures . . . 32
  - 5.3.2 Hyperparameters . . . 33

6 Results and Discussion **34**
6.1 Comparison to Baseline Methods 34
6.2 Loss Curve Analysis 35
6.3 Latent Space Visualization 37
6.4 Confusion Map Insights 39
6.5 Impact of Cluster Number k 40
6.6 Performance on Different VAE Structure 41
6.7 Effect of Exclusive Lasso 41

7 Conclusion **42**

A Symmetric Non-negative Matrix Factorization **47**

B Optimization Methods of Non-negative Matrix Factorization **48**

C Further Experimental Results **49**

1 Introduction

Clustering is a classical unsupervised learning problem in machine learning that serves as a important step in many data mining tasks. By learning the intrinsic patterns within data, clustering technique aims to partition samples into distinct groups such that similar samples belong to the same cluster while samples from different clusters exhibit dissimilarity. Clustering techniques have found significant applications across diverse domains, including image processing, natural language processing, and bioinformatics [8].
While traditional clustering methods have achieved remarkable success, the development of internet services has led to a huge increase in data scale and the growing research has focused on unstructured, high-dimensional data. Traditional clustering approaches face severe limitations and challenges due to computational efficiency constraints, sensitivity to initialization and outliers, and the curse of dimensionality. With the continuous advancement of deep learning techniques and their successful application in representation learning, researchers have begun to employ these techniques into deep clustering frameworks [2]. By combining representation learning with clustering methods, these approaches can extract compact, structured, and interpretable representations from complex high-dimensional data as input for clustering algorithms, thereby significantly enhancing clustering performance.



Figure 1: Five classic 2-D synthetic datasets and the 2-D latent representation learned by an Autoencoder.

However, a lot of evidence suggests that learning a representation that is truly cluster friendly remains challenging, even when the objective appears straightforward. Figure 1 shows the original five classic 2-D synthetic datasets and latent space of a prototypical baseline model autoencoder. It demonstrates that the generated two-dimensional latent manifold still preserves much of the original curvature. A standard autoencoder (AE) trained exclusively with mean-squared-error (MSE) reconstruction loss preserves local geometric structure, yet it scarcely separates clusters. Minimizing reconstruction error alone is therefore inadequate for learning representations that are conducive to clustering. Euclidean distance alone is insufficient to yield fully separable clusters. Similar behavior may occur in more complex image and text datasets, where the shape of the latent space is typically governed by multiple interacting factors.
To obtain a latent space that facilitates clustering, an auxiliary clustering objective must be introduced. A widely adopted strategy is to incorporate a k-means loss computed in the latent space into the overall objective. The core idea of

adding the k-means loss on the latent space of an autoencoder is that the low-dimensional representations should be distributed around their corresponding cluster centroids.
Frameworks that couple AE representation learning with k-means clustering have demonstrated strong effectiveness and performance gains in a variety of domains. However, the conventional k-means loss depends on directly optimizing discrete cluster-assignment variables, which prevents synchronous updates within an end-to-end model trained by gradient descent.

1.1 Problem Formulation

This study addresses the inherent limitations of traditional clustering methods by proposing an integrated deep clustering framework. The key idea is to integrated the feature extraction capabilities of Variational Autoencoders (VAEs) for representation learning with the clustering capabilities of Non-negative Matrix Factorization (NMF). Mathematically, the objective is to jointly optimize the representation embedding and clustering assignment, which is formulated as a unified optimization problem. The deep generative component aims to map high-dimensional input data into a structured latent space, while the NMF clustering component seeks to factorize the representation into interpretable clusters. By combining these two components, we aim to implement an end-to-end clustering model, and obtain an interpretable clustering representation through the joint optimization. To achieve this goal, this paper primarily addresses the following three sub-problems:

1. How does k-means implemented as a matrix-factorization relaxation perform on clustering tasks? In particular, how does it differ from the classic k-means algorithm with respect to accuracy and robustness, and can it scale to different data structures and high-dimensional datasets?

2. How can the k-means clustering loss, expressed via matrix factorization, be integrated into representation-learning models to induce a latent space that is favorable for clustering?

3. How can a VAE-based representation model be jointly optimized with a clustering loss, enabling end-to-end mini-batch updates that yield real-time, interpretable, and consistently superior clustering performance at each iteration?

1.2 Main Results

The proposed method was evaluated using widely recognized benchmarks including MNIST, Fashion-MNIST and REUTERS-10k datasets. Experimental results demonstrate superior clustering performance compared to traditional clustering techniques, showing consistent improvements in clustering metrics such as accuracy, normalized mutual information (NMI), adjusted Rand index (ARI), purity, and F1-measure. Latent space visualizations further confirmed the model's ability to generate well-separated and interpretable clusters.
Additionally, the analysis using confusion maps provided valuable insights into clustering dynamics, pinpointing areas requiring further refinement. Investigations on finding the optimal number of clusters confirmed the capacity to effectively balance model complexity and interpretability.

1.3 Contributions

This paper proposes an end-to-end mini-batch joint optimization framework that integrates VAEs with NMF for deep clustering. Our approach enables simultaneous optimization of representation learning and clustering, leveraging the structured latent space from VAEs and the explicit interpretability of NMF. The resulting NMF indicator matrix provides inherently interpretable cluster assignments, ideally approaching a strict one-hot structure. However, it is challenging to perform end-to-end optimization under binary constraints directly due to their discrete and non-differentiable nature. To overcome this issue, we introduce a continuous relaxation strategy that combines a binary penalty term with NMF. Specifically, we propose an improved PRIMP-based proximal operator that offers a closed-form update, jointly optimizing the representation-learning and clustering modules. The operator preserves the non-negativity and sparsity of the NMF indicator matrix, yielding interpretable cluster results directly. Additionally, the binary constraints on cluster assignments are relaxed into a sparsity-promoting regularization term embedded within the NMF optimization process. This relaxation allows the model to approximate discrete binary solutions in a continuous and differentiable manner. Furthermore, it enables true end-to-end training: the entire model is updated in a single forward–backward pass, guaranteeing convergence while remaining computationally efficient and scalable. Overall, this study bridges the long-standing gap between deep representation learning and clustering. The primary contributions of this paper are summarized as follows:

1. We Propose a deep clustering framework combining the representation learning capabilities of VAEs and the clustering capabilities of NMF inspired by the k-means objective. Through the implementation of NMF, we get the differentiable clustering loss which is equivalent to k-means objective.

2. We introduce a joint optimization strategy that leverages alternating optimization and proximal gradient descent combined with an Exclusive Lasso regularizer, enabling the model to simultaneously learn a clustering-friendly latent space and produce up-to-date, sparse, and interpretable clustering results, while ensuring efficient training and robust convergence.

3. We conduct comprehensive experiments on benchmark datasets (MNIST, Fashion-MNIST and REUTERS-10k), demonstrating significant performance gains over traditional clustering methods.

4. We providing detailed latent space visualizations and clustering dynamics analysis, thereby validating the interpretability of the proposed approach.

1.4 Outline

The remainder of this thesis is outlined below. In Chapter 2 Background, the core concepts related to this work will be introduced, specifically traditional clustering algorithms and deep clustering methods. Moreover, related relevant literature will be discussed. In Chapter 3 Related Work, the theoretical foundations of VAE and NMF are discussed as well as the mathematical derivations and optimization methods. Additionally, binary constraints and regularization

techniques for clustering are presented. We will cover each component of the proposed model architecture in Chapter 4 Method. We will detail the objective function formulation and training strategies including alternating optimization and proximal gradient descent procedures to achieve our proposed model. We introduce the experiments in Chapter 5 Experiments, including dataset descriptions, evaluation metrics, and implementation details. In Chapter 6 Results and Discussion, we analyze the performance comparisons with baseline methods, examine loss curves and latent space visualizations, provide confusion map insights, investigate the impact of cluster numbers, and compare different VAE architectures. In Chapter 7 Conclusion, we summarize this work and present some directions for future work.

2 Background

Clustering, a fundamental unsupervised machine learning technique, aims to uncover the underlying structure and latent patterns within data.Traditional clustering algorithms, such as k-means, hierarchical clustering, and density-based methods, have been widely studied and successfully applied across numerous domains. These classical approaches typically perform well when dealing with relatively small datasets with clear structure.

However, traditional clustering methods encounter significant challenges when faced with large-scale, high-dimensional, or unstructured data, such as textual documents, images, videos, or audio. These methods often rely on manually crafted features or raw data for partitioning, making it difficult to find the underlying structure or complex semantic relationships present in the raw data. With the significant development made by deep representation in unstructured and high-dimensional data, deep clustering methods have been increasingly explored to capture complex relationships between samples for better clustering performance. Deep clustering methods, therefore, combine the advantages of deep representation learning with clustering algorithms, enabling them to effectively discover more structured and interpretable latent patterns within complex data.

Generally, deep clustering consists of two essential components: a representation learning module and a clustering module. The representation learning module uses deep neural networks, such as Autoencoders, VAEs, or convolutional networks, to project data into a structured latent space. The intrinsic relationships among samples are easier to distinguish after the projection. The clustering module, on the other hand, utilizes techniques such as k-means, Gaussian Mixture Models (GMM), or NMF, to partition the learned latent representations into distinct groups. Meanwhile, the information exchange between two module is the most important part in deep clustering, which ensures the representation module learns clustering-friendly embeddings and the clustering module simultaneously adapts to the evolving representations. This iterative and interactive process facilitates the learning of discriminative and interpretable embeddings, significantly improving clustering performance.

In summary, this section first provides an overview of traditional clustering methods based on partition, highlighting their advantages and limitations. Then, we introduce the key components of deep clustering: representation learning and clustering modules, discussing their interaction during the joint optimization

process. Throughout this discussion, critical technologies and methodological innovations are highlighted, underscoring the advantages of deep clustering in addressing traditional clustering challenges.

2.1 Traditional Clustering

By automatically partitioning the dataset, clustering assigns similar data points to the same cluster, while different data points are assigned to different clusters. The input to a clustering algorithm is the raw dataset and the output is clustering labels. The traditional clustering methods can be broadly categorized into the following types: partition-based methods, density-based methods, hierarchical methods, and model-based methods. In this paper, we mainly focus on the k-means and its variants, which is belong to the category of partition-based methods. We incorporate NMF as the clustering module, leveraging its ability to produce sparse and interpretable indicators of latent cluster structure. In this section, we proved that when an orthogonality constraint is imposed on the indicator matrix, NMF can be regarded as a clustering method that is mathematically equivalent to k-means clustering. For completeness we give the full derivation of the orthogonality-relaxed formulation and its multiplicative updates in Appendix.

2.1.1 k-means Clustering

Partition-Based methods partition the dataset by predefining a fixed number of clusters and iteratively updating the center of each cluster to optimize the assignment. A classic example is k-means clustering [21], which works effectively for partitioning data into spherical clusters in convex datasets. However, this type of method has several limitations, including the requirement of predefining the number of clusters, sensitivity to the initialization of centroids, and susceptibility to outliers. Additionally, it struggles to achieve a reasonable results when applied to non-convex dataset.
The k-means clustering objective function quantifies the variance within clusters and can be expressed as follows

$$\mathcal{L}_{K\text{-means}} = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - c_k\|^2 \tag{1}$$

$$= \sum_{k=1}^{K} \sum_{i \in C_k} \left(\|x_i\|^2 - 2x_i^\top c_k + \|c_k\|^2 \right), \tag{2}$$

where c_k is the centroid of cluster C_k, defined as $c_k = \frac{1}{n_k} \sum_{i \in C_k} x_i$, and n_k represents the number of data points in cluster C_k. The term $\sum_{k=1}^{K} \sum_{i \in C_k} \|x_i\|^2$ is constant with respect to the centroids and can be denoted by S

$$\mathrm{S} = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i\|^2. \tag{3}$$

Using the definition of the centroid, the cross-term can be expanded as

$$\sum_{i \in C_k} x_i^\top c_k = \sum_{i \in C_k} x_i^\top \left(\frac{1}{n_k} \sum_{j \in C_k} x_j \right) = \frac{1}{n_k} \sum_{i \in C_k} \sum_{j \in C_k} x_i^\top x_j. \tag{4}$$

Similarly, the norm squared of the centroid c_k is given by

$$\|c_k\|^2 = \left\| \frac{1}{n_k} \sum_{i \in C_k} x_i \right\|^2 = \frac{1}{n_k^2} \sum_{i \in C_k} \sum_{j \in C_k} x_i^\top x_j. \tag{5}$$

Substituting these expressions back into Eq. (2), we obtain

$$\begin{aligned} \mathcal{L}_{K\text{-means}} &= \mathrm{S} - 2 \sum_{k=1}^{K} \frac{1}{n_k} \sum_{i \in C_k} \sum_{j \in C_k} x_i^\top x_j + \sum_{k=1}^{K} \frac{n_k}{n_k^2} \sum_{i \in C_k} \sum_{j \in C_k} x_i^\top x_j \\ &= \mathrm{S} - \sum_{k=1}^{K} \frac{1}{n_k} \sum_{i \in C_k} \sum_{j \in C_k} x_i^\top x_j, \end{aligned} \tag{6}$$

where the simplification comes from consolidating the terms involving pairwise interactions within each cluster.

2.1.2 Non-negative Matrix Factorization

Let the input data matrix $X = (x_1, \cdots, x_n)^T$ represents the n data points in d dimensions. Through NMF, we factorize X into two matrices

$$X \approx YC, \tag{7}$$

where $X \in \mathbb{R}^{n \times d}$, $Y \in \mathbb{R}_+^{n \times k}$, and $\in \mathbb{R}_+^{k \times d}$. The matrix Y is defined as the hard-class indicator matrix where each column represents a cluster and each row represents a data point. If a data point belongs to a specific cluster, the corresponding entry is 1, otherwise it is 0. For example, if the i-th data point belongs to the k-th cluster, then $Y_{ik} = 1$

$$Y_{ik} = \begin{cases} 1 & \text{if } X_i \in C_k \\ 0 & \text{otherwise} \end{cases} \tag{8}$$

This exclusivity constraint ensures that each data point is uniquely assigned to a cluster, without overlap between clusters in terms of their indicator representations. The matrix C is defined as the centroid matrix. Each row k of C, corresponding to c_k, represents the centroid of the k-th cluster. Generally, the rank of the matrices Y and C is much lower than the rank of X (i.e., $k \ll \min(d, n)$). Taking NMF as an optimization problem, the objective function can be written as

$$\min_{Y \geq 0, C \geq 0} \mathcal{L}_{\text{NMF1}} = \|X - YC\|^2. \tag{9}$$

However, this exclusivity constraint on Y is too strict and usually difficult to solve. In this case, the optimization problem is typically NP-hard. We can relax this constraint by transforming it into an orthogonal constraint, converting the discrete optimization problem into a continuous optimization problem. We introduce a new indicator matrix H

$$H = (\mathbf{h}_1, \ldots, \mathbf{h}_k), \quad \mathbf{h}_k^T \mathbf{h}_\ell = \delta_{k\ell}, \tag{10}$$

where

$$\mathbf{h}_k = \left(0, \ldots, 0, \overbrace{1, \ldots, 1}^{n_k}, 0, \ldots, 0\right)^T / \sqrt{n_k}, \tag{11}$$

and $\delta_{k\ell}$ represents the Kronecker delta function which is defined as follows

$$\delta_{k\ell} = \begin{cases} 1 & \text{if } k = \ell, \\ 0 & \text{if } k \neq \ell. \end{cases} \tag{12}$$

$\mathbf{h}_k^T \mathbf{h}_\ell = \delta_{k\ell}$ indicates that the vectors $\mathbf{h}_k$ and $\mathbf{h}_\ell$ are orthogonal, with their inner product being 1 when $k = \ell$ and 0 when $k \neq \ell$. This ensures the independence and orthogonality between the vectors. Therefore, $H^T H = I$ indicating that the clusters are mutually orthogonal in this representation. This orthogonality constraint can also ensure that each data point is assigned to only one cluster. Now the objective function becomes

$$\min_{Y \geq 0, C \geq 0} \mathcal{L}_{\text{NMF2}} = \|X - HC\|^2, \quad \text{s.t. } H^T H = I. \tag{13}$$

To simplify this objective function, applying the trace operator, we obtain

$$\begin{aligned} \mathcal{L}_{NMF2} &= \text{Tr}((X - HC)^T (X - HC)) \\ &= \text{Tr}(X^T X - 2C^T H^T X + C^T H^T HC) \end{aligned} \tag{14}$$

The derivative of $\mathcal{L}_{\text{NMF2}}$ with respect to C is given by

$$\frac{\partial \mathcal{L}_{\text{NMF2}}}{\partial C} = -2H^T X + 2H^T HC \tag{15}$$

By setting the derivative to zero, given the orthogonality condition $H^T H = I$, we obtain

$$C = H^T X \tag{16}$$

Substituting $C = H^T X$ into Eq. (14), we get

$$\begin{aligned} \mathcal{L}_{\text{NMF2}} &= \text{Tr}(X^T X - 2(H^T X)^T H^T X + (H^T X)^T (H^T X)) \\ &= \text{Tr}(X^T X - H^T X X^T H) \end{aligned} \tag{17}$$

Since $X^T X$ is a constant, the optimization problem becomes

$$\max_{H \geq 0} \text{Tr}(H^T X X^T H), \quad \text{s.t. } H^T H = I. \tag{18}$$

2.1.3 Relationship between k-means Clustering and NMF

Lee and Seung [15] discussed the relationship between NMF and Vector Quantization which is K-means clustering. The research by Li and Ma [18] demonstrates that matrix factorization exhibits clustering effects. The clustering aspect of NMF is also utilized in a document clustering approach which was proposed by Xu *et al.* [35]. The experiments conducted by Li *et al.* that demonstrate the clustering capabilities of NMF and a formal proof of this theorem is provided in their work [17].

By using hard-class indicator matrices Y with strict orthogonality constraints, it can be demonstrated that NMF is completely equivalent to k-means clustering. Based on the definitions of the indicator matrix Y and the centroid matrix C, YC forms a new centroid matrix, with the i-th row representing the centroid corresponding to x_i. Therefore, Eq. (9) is completely equivalent to Eq. (1), which means NMF is equivalent to k-means clustering.
Considering the indicator matrix under orthogonality constraints, NMF is also essentially identical to K-means clustering.

Theorem 1. *NMF of $X \approx HC$ with orthogonality constraints is equivalent to kernel k-means clustering.*

Proof. Based on the definition of indicator matrix H ,We can obtain

$$(H^T X)^T H^T X = \sum_{k=1}^{K} \frac{1}{n_k} \sum_{i \in C_k} \sum_{j \in C_k} x_i^\top x_j \tag{19}$$

As a result, Eq. (6) can be written as

$$\mathcal{L}_{K\text{-means}} = \mathrm{S} - \mathrm{Tr}(H^T X X^T H). \tag{20}$$

Minimizing this loss function is equivalent to Eq. (18). Therefore, NMF with orthogonal constraints and K-means clustering are equivalent. □

2.2 Representation Module

Deep representation learning is used for feature extraction, with the objective of learning nonlinear, low-dimensional embeddings from the high-dimensional raw data. By capturing hierarchical and abstract features, deep representation learning can effectively map high-dimensional data into a compact and expressive low-dimensional latent space while preserving the intrinsic structure and underlying patterns of the data. The learned feature representations are more discriminative and conducive to subsequent clustering tasks, enabling better analysis and understanding of the data. The main categories of deep representation learning methods are as follows [26, 38]:

2.2.1 Feedforward Network

In the field of deep clustering, feedforward neural networks have been widely employed as the fundamental network architecture, with the objective of optimizing specific clustering loss functions [22]. Feedforward neural networks can be mainly categorized into two types: fully-connected neural networks (FCNs) and convolutional neural networks (CNNs).
FCNs, also know as Multi-Layer Perceptrons (MLPs), consist of an input layer, multiple hidden layers, and an output layer. Each neuron in one layer is fully connected to all neurons in the subsequent layer, with each connection having its own independent weight. This architecture ensures that input features can be comprehensively propagated to the next layer. However, as the data dimensionality and the number of network layers increase, the parameters grow rapidly, leading to high computational complexity and a greater risk of overfitting.
CNNs are deep learning models specifically designed to process data with a grid-like structure. A key feature of CNNs is their use of convolutional layers,

which employ kernels that slide over the input data to extract local features and create feature maps. Each neuron in a layer is connected to only a limited number of neurons in the previous layer. By using the the same convolutional kernels across positions, CNNs significantly reduces the number of learnable parameters. This enhances computational efficiency and reduces the risk of overfitting. The local connectivity and parameter sharing in CNNs allow them to capture translation-invariant features and handle spatial variations in the input data effectively.

2.2.2 Autoencoder

An autoencoder (AE) [24] is an unsupervised deep learning model that improves from traditional feedforward networks. It comprises an encoder and a decoder. The encoder is to maps the input data to a latent space, extracting the features of the original data into a low-dimensional embedding. This process allows the AE to learn a compact and meaningful representation of the input data. The decoder takes the latent space generated by encoder and try to reconstruct the original input data. The decoder maps the low-dimensional embedding back to the same dimensions as the input. The objective of the AE is to minimize the reconstruction loss, which measures the difference between original input and the reconstructed output, during the training process.
In deep clustering problems, the training process of AE mainly consists of two stages. In the first stages, AE is pretrained to minimize the reconstruction loss, which is same to the regular AE. During this process, we often use mean squared error (MSE) as the loss function

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} \|x_i - \hat{x}_i\|^2 , \tag{21}$$

where x_i represents the original input, $\hat{x}_i$ denotes the reconstructed output, and N is the total number of samples. In the second stage, AE is fine-tuned by a clustering loss to optimize the performance on a specific clustering task. The choice of clustering parameters can significantly influence the clustering results and the model's overall performance.

2.2.3 Deep Generative Model: VAE

Generative methods for representation learning are based on the assumption that data x are generated from latent variable z and deduce the posterior distribution $q(z|x)$ from the data. Variational Autoencoder (VAE) [11] is one of the most typical generative models that extends the concept of AE by introducing a probabilistic distribution in the latent space, which is typically assumed to be a Gaussian distribution. The encoder in VAE maps the input data to a latent variable space, learning the probability distribution of the data. It outputs the mean μ and variance σ^2 of the latent representation instead of directly producing a latent representation as in an AE. By generating a distribution rather than a single point, VAE can maintain a continuous and structured latent space. The decoder takes the latent variable z as the input to generate the reconstructed data, where z was sampled from a Gaussian distribution parameterized as

$$z \sim \mathcal{N}(\mu, \sigma^2), \tag{22}$$

However, the direct sampling operation is non-differentiable, which prevents backpropagation training in the train stage. To address this issue, VAE introduces the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \tag{23}$$

where $\epsilon \sim \mathcal{N}(0, I)$ represents random noise sampled from the standard normal distribution, and $\odot$ represents element-wise product. This trick represents z as a linear combination of μ and σ, effectively decoupling the stochastic sampling from the deterministic parts of the neural network. It ensures the sampling process is differentiable with respect to the network parameters, enabling proper network training.

VAE optimizes the Evidence Lower Bound (ELBO) as its objective function, which approximates the intractable log-likelihood of the data. The ELBO consists of two main components: the reconstruction loss and the KL divergence. The reconstruction loss in VAE measures the similarity between the original data and the reconstructed data generated from the latent variable, which is same as that in AE. The KL divergence measures the similarity between posterior distribution $q(z|x)$ and the assumed prior distribution $p(z)$. The loss function of VAE can be expressed as

$$\begin{aligned} \mathcal{L}_{\text{VAE}} &= \mathcal{L}_{\text{recon}} - \mathcal{L}_{\text{KL}} \\ &= \mathbb{E}_{q(z|x)}\big[\log p(x|z)\big] - D_{\text{KL}}\big(q(z|x)\|p(z)\big), \end{aligned} \tag{24}$$

where $\mathbb{E}_{q(z|x)}[\log p(x|z)]$ represents the reconstruction loss, typically implemented as MSE for continuous data or Cross-Entropy Loss for discrete data. KL divergence term $D_{\text{KL}}(q(z|x)\|p(z))$ regularizes the latent space by penalizing deviations of the approximate posterior $q(z|x)$ from the prior distribution $p(z)$, typically chosen as a standard Gaussian, which prevents the model from overfitting the latent variable distribution to the training data.

Compared to AE, VAE has a significant advantage in representation learning due to their latent space regularization, which is achieved by applying a KL divergence term with a prior distribution. This regularization ensures continuity in the latent space, which makes the smoothness of extracted features. In the latent space, points that are close to each other correspond to similar features, ensuring that transitions within the space lead to gradual and meaningful changes in extracted features. This property enhances the interpretability of the model, providing a more structured and continuous representation that aids in downstream tasks.

2.3 Variational Autoencoder: Detailed derivation and implementation

In the proposed method in this paper, we adopt VAE as the representation learning module. As described in Section 2.2.3, the VAE assumes that the observed data x is generated from the conditional distribution $p_\theta(x \mid z)$, which corresponds to the decoder of the VAE, where z is a continuous latent variable generated from a prior distribution $p_\theta(z)$. When using VAE for representation learning, the objective is to get the latent variable z from given observed data x which is denoted as $p_\theta(z \mid x)$. This probability distribution represents the

encoder of the VAE. According to Bayes' theorem, we can obtain

$$p_\theta(z \mid x) = \frac{p_\theta(x \mid z) p_\theta(z)}{p_\theta(x)}, \tag{25}$$

where the distribution of x is

$$p_\theta(x) = \int p_\theta(z) p_\theta(x \mid z)\, dz. \tag{26}$$

In this case, the multidimensional integral computation over the latent variable z can be challenging. To address this issue, we introduce a neural network with parameters ϕ to define a distribution q_ϕ, which aims to approximate the posterior distribution $p_\theta(z \mid x)$).

2.3.1 Loss Function

To measure the similarity between the two distributions, we use the KL divergence as the loss function. By definition, we can obtain

$$\begin{aligned} D_{\mathrm{KL}}\left(q_\phi(z \mid x) \| p_\theta(z \mid x)\right) &= -\sum_z q_\phi(z \mid x) \log\left(\frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\right) \\ &= -\sum_z q_\phi(z \mid x) \log\left(\frac{p_\theta(x, z)/p_\theta(x)}{q_\phi(z \mid x)}\right) \\ &= -\sum_z q_\phi(z \mid x) \left[\log\left(\frac{p_\theta(x, z)}{q_\phi(z \mid x)}\right) - \log(p_\theta(x))\right], \end{aligned} \tag{27}$$

since $\log(p_\theta(x))$ is independent of z, it is a constant and can be moved to the left side of the equation. Consequently, we can obtain

$$\begin{aligned} \log p_\theta(x) &= D_{\mathrm{KL}}(q_\phi(z \mid x) \| p_\theta(z \mid x)) + \sum_z q_\phi(z \mid x) \log\left(\frac{p_\theta(x, z)}{q_\phi(z \mid x)}\right) \\ &= D_{\mathrm{KL}}(q_\phi(z \mid x) \| p_\theta(z \mid x)) + \mathcal{L}(\theta, \phi; x), \end{aligned} \tag{28}$$

where $\mathcal{L}(\theta, \phi; x)$ is referred as the variational lower bound, also known as the Evidence Lower Bound (ELBO). Since the KL divergence is a non-negative term, optimizing the loss function can be transformed into the maximization of the ELBO. Here, we expand the ELBO and obtain

$$\begin{aligned} \mathcal{L}(\theta, \phi; x) &= \sum_z q_\phi(z \mid x) \log\left(\frac{p_\theta(x, z)}{q_\phi(z \mid x)}\right) \\ &= \sum_z q_\phi(z \mid x) \log\left(\frac{p_\theta(x \mid z) p_\theta(z)}{q_\phi(z \mid x)}\right) \\ &= \sum_z q_\phi(z \mid x) \left[\log(p_\theta(x|z)) + \log\left(\frac{p_\theta(z)}{q_\phi(z \mid x)}\right)\right] \\ &= \underbrace{\mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right]}_{\text{Reconstruction loss}} - \underbrace{D_{\mathrm{KL}}\left(q_\phi(z \mid x) \,\|\, p_\theta(z)\right)}_{\text{KL regularization loss}}, \end{aligned} \tag{29}$$

where the first term is the reconstruction loss, and the second term is the KL regularization loss. Here, we have completed the derivation of Eq. (24), which is loss function of VAE.

2.3.2 Optimization Strategy

In order to optimize $L(\theta, \phi; x)$, that is, to maximize the ELBO, we need to compute the gradient with respect to θ. However, according to Eq. (26), $p_\theta(x)$ is obtained by integrating over the latent variable z. In the case of high dimensional or nonlinear conditional distributions, this integration can be challenging and hard to get a simple closed-form expression. Therefore, $log(p_\theta(x))$ is difficult to express as a differentiable explicit function, making it possible to directly optimize the loss function by using maximum likelihood estimation.
To address this issue, we can use the naive Monte Carlo gradient estimator, which can obtain an unbiased estimate of the gradient by sampling and multiplying by the "score function" [23]. This optimization process involves drawing samples $\{x^{(n)}\}$ from the distribution $p_\theta(x)$ and then computing the average of the function evaluated at these samples. Firstly, we assuming an $f(z)$ that is independent of ϕ

$$f(z) = \log(p_\theta(x|z)) + \log\left(\frac{p_\theta(z)}{q_\phi(z|x)}\right), \tag{30}$$

then we use the naive Monte Carlo gradient estimator to compute the gradient of $f(z)$ under $q_\phi(z)$ with respect to ϕ and express it in the form of expectation

$$\begin{aligned}
\nabla_\phi \mathbb{E}_{q_\phi(z)}\left[f(z)\right] &= \int \nabla_\phi [q_\phi(z)] f(z) dz \\
&= \int q_\phi(z) \nabla_\phi \log q_\phi(z) f(z) dz \\
&= \mathbb{E}_{q_\phi(z)}\left[f(z)\, \nabla_\phi \log q_\phi(z)\right] \qquad (31) \\
&\simeq \frac{1}{L} \sum_{\ell=1}^{L} \left[f\left(z^{(\ell)}\right) \nabla_\phi \log q_\phi\left(z^{(\ell)}\right)\right], \qquad (32)
\end{aligned}$$

where $z^{(i)} \sim q_\phi(z|x^{(i)})$ and $\ell \in L$ denotes the ℓ-th Monte Carlo sampling. This estimator requires a large number of samples to converge, which can result in high variance. Consequently, when train with high-dimensional latent spaces or complex variational distributions, the training process may have the instability problem and slow convergence rates.
To address the issue of high variance, Kingma and Welling [11] proposed an unbiased, differentiable, and scalable estimator for the ELBO in variational inference, which is known as Stochastic Gradient Variational Bayes (SGVB). In the naive Monte Carlo gradient estimator, we make an assumption that the $f(z)$ is independent of phi.By differentiating $f_\phi(z)$ with respect to ϕ, we can compute gradient without this assumption. But this differentiation might not be computationally tractable, because z is directly sampled from a distribution parameterized by ϕ. Here we use the reparameterization trick described in Section 2.2.3. Through Eq. (23), the expectation can be written as

$$\mathbb{E}_{p_\phi(z)}\left[f(z)\right] = \mathbb{E}_{p(\epsilon)}\left[f(g_\phi(\epsilon, x))\right], \tag{33}$$

Hence, when we take the gradient respect to ϕ, we can apply the chain rule to

backpropagate. As a result, the gradient of the expectation can be written as

$$\begin{aligned}\nabla_\phi \mathbb{E}_{p_\phi(z)}\big[f\big(z\big)\big] &= \nabla_\phi \mathbb{E}_{p(\epsilon)}\left[f\left(g_\phi(\epsilon, x)\right)\right] \\ &= \mathbb{E}_{p(\epsilon)}\left[\nabla_\phi f\left(g_\phi(\epsilon, x)\right)\right] \\ &\simeq \frac{1}{L}\sum_{\ell=1}^{L} \nabla_\phi f(g_\phi(\epsilon^{(\ell)}, x)). \end{aligned} \tag{34}$$

In this case, the ELBO can be expressed as

$$\mathcal{L}(\theta, \phi; x) = \frac{1}{L}\sum_{\ell=1}^{L}\left[\log p_\theta(x, z^\ell) - \log q_\phi(z^\ell \mid x)\right]. \tag{35}$$

In VAE, it is common to assume that $q_\phi(z|x)$ follows a multivariate Gaussian distribution $\mathcal{N}(\mu_\phi(x), \sigma^2_\phi(x))$, and $p_\theta(z)$ follows a standard normal distribution N(0, I). In this case, the KL divergence term in ELBO can has a simple closed-form expression

$$\begin{aligned}-D_{\mathrm{KL}}\big(q_\phi(z)\|p_\theta(z)\big) &= -D_{\mathrm{KL}}\big(\mathcal{N}(\mu, \sigma^2)\|\mathcal{N}(0, 1)\big) \\ &= -\int \mathcal{N}(z; \mu, \sigma^2)\log\frac{\mathcal{N}(z; \mu, \sigma^2)}{\mathcal{N}(z; 0, 1)}dz \\ &= \int \mathcal{N}(z; \mu, \sigma^2)\log\mathcal{N}(z; 0, 1)dz \\ &- \int \mathcal{N}(z; \mu, \sigma^2)\log\mathcal{N}(z; \mu, \sigma^2)dz. \end{aligned} \tag{36}$$

We can compute these two integrals separately. Based on the probability density function of the normal distribution $f_{\mathbf{normal}}(z) = \frac{1}{\sigma\sqrt{2\pi}}e^{-\frac{1}{2}\left(\frac{z-\mu}{\sigma}\right)^2}$,the first term can be written as

$$\begin{aligned}\int \mathcal{N}(z; \mu, \sigma^2)\log\mathcal{N}(z; 0, I)dz &= \int \mathcal{N}(z; \mu, \sigma^2)\left(-\frac{1}{2}z^2 - \frac{1}{2}\log(2\pi)\right)dz \\ &= -\frac{1}{2}\int \mathcal{N}(z; \mu, \sigma^2)z^2 dz - \frac{J}{2}\log(2\pi) \\ &= -\frac{1}{2}\mathbb{E}_{z\sim\mathcal{N}(z;\mu,\sigma^2)}[Z^2] - \frac{J}{2}\log(2\pi) \\ &= -\frac{1}{2}\left(\mathbb{E}_{z\sim\mathcal{N}(z;\mu,\sigma^2)}[Z]^2 + \mathrm{Var}(Z)\right) - \frac{J}{2}\log(2\pi) \\ &= -\frac{1}{2}\sum_{j=1}^{J}(\mu_j^2 + \sigma_j^2) - \frac{J}{2}\log(2\pi), \end{aligned} \tag{37}$$

where J is the dimensionality of z. The second term can be written as

$$
\begin{aligned}
&\int \mathcal{N}(z;\mu,\sigma^2)\log\mathcal{N}(z;\mu,\sigma^2)dz \\
&= -\frac{1}{2}\int \mathcal{N}(z;\mu,\sigma^2)\frac{(z-\mu)^2}{\sigma^2}dz - \frac{J}{2}\log(2\pi) - \frac{1}{2}\sum_{j=1}^{J}\log(\sigma_j^2) \\
&= -\frac{J}{2}\log(2\pi) - \frac{1}{2}\sum_{j=1}^{J}\log(\sigma_j^2) - \frac{1}{2}\mathbb{E}_{z\sim\mathcal{N}(z;\mu,\sigma^2)}\left[\frac{(z-\mu)^2}{\sigma^2}\right] \\
&= -\frac{J}{2}\log(2\pi) - \frac{1}{2}\sum_{j=1}^{J}\log(\sigma_j^2) - \frac{1}{2}\left(\mathbb{E}_{z\sim\mathcal{N}(z;\mu,\sigma^2)}\left[\frac{(z-\mu)^2}{\sigma^2}\right] + \mathrm{Var}\left(\frac{z-\mu}{\sigma}\right)\right) \\
&= -\frac{J}{2}\log(2\pi) - \frac{1}{2}\sum_{j=1}^{J}(1+\log\sigma_j^2) \qquad (38)
\end{aligned}
$$

By substituting the expansion of these two terms back into Eq. (36), we obtain

$$
-D_{\mathrm{KL}}\big(q_\phi(z)\|p_\theta(z)\big) = \frac{1}{2}\sum_{j=1}^{J}\left(1+\log(\sigma_j^2)-\mu_j^2-\sigma_j^2\right). \qquad (39)
$$

Having derived the KL term in closed form, we next consider the reconstruction term $E_{q_\phi(z|x)}[\log(p_\theta(x|z))]$. For continuous input x, it is common to assume that $p_\theta(x \mid z)$ follows a Gaussian distribution $\mathcal{N}(f_\theta(z), \sigma^2 I)$, where $f_\theta(z)$ is the decoder network, and σ is a fixed scale parameter that controls the reconstruction uncertainty. Computing the log-likelihood under this Gaussian distribution yields:

$$
\log p_\theta(x \mid z) = C - \frac{1}{2\sigma^2}\|x - f_\theta(z)\|^2, \qquad (40)
$$

where C $= -\frac{D}{2}\log(2\pi\sigma^2)$ is a constant term, with D being the dimensionality of x. Consequently, maximizing of the log-likelihood is equivalent to minimizing the MSE$\|x - f_\theta(z)\|^2$.
Combining the above KL term and the reconstruction term, we arrive at the closed-form of full ELBO for our VAE

$$
\begin{aligned}
\mathcal{L}(\theta,\phi;x) &= \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] - D_{\mathrm{KL}}\big(q_\phi(z \mid x)\,\|\,p_\theta(z)\big) \\
&= \mathbb{E}_{q_\phi(z|x)}\left[C - \frac{1}{2\sigma^2}\|x - f_\theta(z)\|^2\right] + \frac{1}{2}\sum_{j=1}^{J}\left(1+\log(\sigma_j^2)-\mu_j^2-\sigma_j^2\right) \qquad (41) \\
&\simeq -\frac{1}{2\sigma^2}E_{q_\phi(z|x)}\left[\|x - f_\theta(z)\|^2\right] + \frac{1}{2}\sum_{j=1}^{J}\left(1+\log(\sigma_j^2)-\mu_j^2-\sigma_j^2\right). \qquad (42)
\end{aligned}
$$

Equivalently, we minimize the negative of the ELBO which serves as our loss function in code implementations.

2.4 Binary Constraints

In clustering methods based on NMF, the assignment matrix Y typically represents the cluster assignment, with each element indicating the likelihood of a data sample belonging to a particular cluster. Generally, to enhance the interpretability and to achieve clear clustering assignments, a binary constraint is applied row-wise to the matrix Y. In case of the hard clustering assignments, this constraint becomes an exclusive binary restriction, enforcing each row of the matrix Y contains only one non-zero element to form a one-hot embedding, thus ensuring each data point only assigned to exactly one cluster. Such exclusive constraint significantly enhance the interpretability and improve both the stability of the optimization process and the sparsity of the resulting solutions. However, this strict binary constraint is challenging for optimization due to its non-differentiability. A practical approach is to use a relaxed continuous binary constraint, gradually tightening it throughout the optimization procedure. This incremental tightening strategy enables numerical optimization methods to progressively approximate the desired binary solution.

2.4.1 Binary Regularization

A common strategy to enforce binary constraints is to add a regularization term into the objective function. The core idea of the binary regularizer is to augment the objective function that derives each element toward 0 or 1 during the optimization, thereby resulting clear cluster assignments.
Among these methods, the Mexican hat function is a widely used binary regularizer, which is defined as $R_{MH}(x) = \frac{\lambda}{2}(x^2 - x)^2 = \frac{\lambda}{2}x^2(1-x)^2, \quad \lambda > 0$ [37]. This regularizer has a clear concave-convex shape with symmetrical double minima at the endpoints 0 and 1, and a local maximum at the midpoint 0.5. The introduction of this regularizer preserves the smoothness of the objective function and ensures the continuous differentiability of the Hessian matrix. However, optimization occasionally gets stuck in suboptimal values and is hard to strictly converge all elements to 0 or 1 due to the vanishing gradients. Another common regularizer is PRIMP [7], defined as $R_{\text{PRIMP}}(x) = \kappa\,(1 - |1 - 2x|)$ for $x \in [0, 1]$, and $+\infty$ otherwise. It introduces infinite barriers outside the unit interval, while inside $[0, 1]$ it forms a triangular profile that peaks at $x = \frac{1}{2}$ with height κ. Consequently, any element that drifts toward the middle region gets a linear penalty, which drives the optimization to settle at the exact 0 or 1. Elastic-Binary(ELB) regularizer, also a widely-used method, defined as $R_{\text{ELB}}(x) = \min\left\{\kappa|x| + \frac{\lambda}{2}x^2, \quad \kappa|x-1| + \frac{\lambda}{2}(x-1)^2\right\}, \quad \kappa, \lambda > 0$. The core idea is to combine two quadratic term centered at 0 and 1, with a linear elastic term $\kappa|x| + \frac{\lambda}{2}x^2$, resulting in a "$W$"-shaped regularizer. This regularizer has clear endpoints by maintaining finite gradients in each region while preserving continuous differentiability. By avoiding the infinite barriers, the variables can wander outside $[0, 1]$ during early iterations yet still converges to 0 or 1. To visualize the differences, Figure 2 plots the penalty landscape of the three regularizes discussed above.

2.4.2 Exclusive Binary Restriction

Exclusive Binary Restriction enforces that each sample is assigned to exactly one cluster, which is used for hard clustering assignments, imposing a stricter

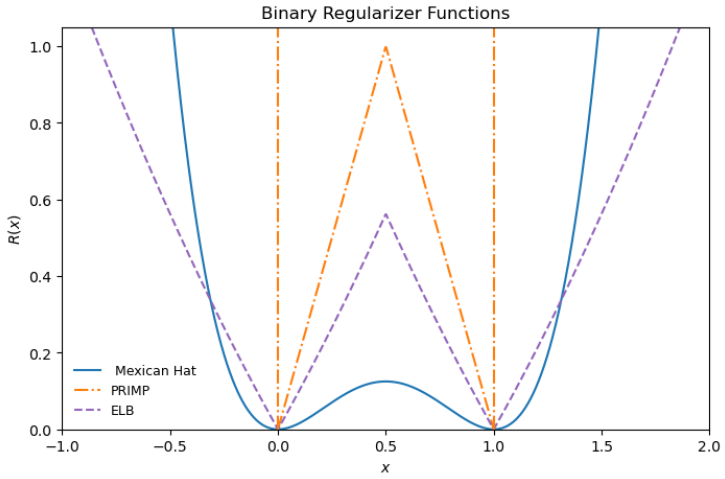


Figure 2: Illustration of three typical binary-regularization functions—Mexican Hat, PRIMP, and ELB. The hyper-parameters are fixed at $\lambda_{\text{Bowl}} = 2$, $\kappa_{\text{PRIMP}} = 1$, and $(\kappa, \lambda)_{\text{ELB}} = (1, 0.5)$, which keeps all curves within the common vertical window in order to more intuitively show the difference between different regularizers.

requirement than the binary regularization. By introducing appropriate regularization terms, the resulting NMF factorization can be driven toward a highly discreet one-hot representation.

Sparsity regularization [10], which is commonly referred to as the ℓ_1 norm, is a widely used method for one-hot representation. The ℓ_1 penalty is defined as $\lambda \sum_i \|\mathbf{Y}_{j,:}\|_1 = \lambda \sum_i \sum_j |Y_{ij}|$. By applying an equal penalty to every non-zero element, the ℓ_1 norm drives many elements to zero rapidly during gradient-based updates, guiding the optimization toward a sparse result. Although ℓ_1 norm encourages each row of the indicator matrix to retain only a few non-zero elements, it may not guarantee exactly one non-zero element per row. Hence, it is often combined with additional constraints to reinforce exclusivity.

Exclusive Lasso [19], also known as $\ell_{1,2}$ norm likewise pushes each row of the indicator matrix toward a one-hot representation. It first takes the ℓ_1 norm of every row and then squares it, so the entries in the same row compete with one another, which is defined as $\sum_i (\sum_j |Y_{ij}|)^2$. By amplifying the cost of having multiple non-zero elements in a single row, Exclusive Lasso provides a continuous relaxation that closely approximates the hard one-hot representation. While the ℓ_1 norm encourages a global sparsity, Exclusive Lasso enforces exclusivity inside one row.

Orthogonal Non-negative Matrix Factorization achieves hard clustering by explicitly driving the indicator matrix toward orthogonality [5]. The approximate orthogonality constraint requires $Y^T Y \approx I$ forcing the columns of Y to become nearly orthonormal. Consequently, each row of Y can exhibit a high activation in at most one dimension. When orthogonality and non-negativity are both satisfied strictly, each row of the indicator matrix can achieve a one-hot representation.

3 Related Work

Clustering is a widely studied field, and numerous algorithms have been proposed to date. In this paper, we review approaches that incorporate an explicit clustering loss into representation learning based on autoencoders, and we briefly highlight the advantages and the differences of these methods. The clustering losses considered here can be divided into two categories: distance–minimization and probability distribution, exemplified by k-means and Gaussian mixture models (GMMs), respectively.

3.1 Distance–minimization Methods

Combining distance-based clustering with deep neural representation learning is a classic method in deep clustering. By introducing ak-means–based clustering loss, the representation module is driven to learn a nonlinear embedding space that is better suited to k-means. The core idea is to encourage each low-dimensional representation to concentrate around its assigned cluster centroid, thereby making the learned features highly compatible with the k-means algorithm. Experiments demonstrate that classical clustering algorithms can be effectively embedded within deep learning frameworks through careful optimization design, achieving superior results compared to applying k-means directly on raw features or pretrained representations. The loss for this family of deep clustering methods can be defined as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{cluster}} = \sum_{i=1}^{N} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 + \lambda \sum_{i=1}^{N} \left\| f(\mathbf{x}_i) - \mathbf{c}_{k(i)} \right\|^2, \tag{43}$$

where $f(\cdot)$ is the encoder of the autoencoder that produces the latent embedding, and $\mathbf{c}_{k(i)}$ denotes the k-th cluster centroid corresponding to $\mathbf{x}_i$in that latent space.

Song et al. [30] are the first to incorporate a k-means clustering loss into auto-encoder training. The optimization follows an alternating strategy. Firstly, with cluster centroids fixed, the auto-encoder parameters are updated via stochastic gradient descent (SGD). Then with the network fixed, the centroids are recomputed from the current latent space. The last step is to reassign samples according to the updated centroids. This approach uses hard assignments without explicit assignment variables: in each iteration, all N samples are used to recompute every cluster center and to reassign cluster labels.

Deep Clustering Network (DCN) [36] extends this idea by explicitly embedding the standard k-means term into the loss:

$$\mathcal{L}_{\text{cluster}} = \sum_{i} \left\| \mathbf{y}_i - \mathbf{C}\mathbf{s}_i \right\|^2, \tag{44}$$

where $\mathbf{C} \in \mathbb{R}^{d \times K}$ is the transposed centroid matrix and $\mathbf{s}_i \in \{0,1\}^K$ is a one-hot indicator vector. During training, a mini-batch scheme is adopted: the loss is computed in a forward pass and then the autoencoder parameters are updated via SGD. With the autoencoder network parameters θ and $\mathbf{C}$ fixed, the indicator

vector of a sample x_i can be updated online as

$$s_{j,i} = \begin{cases} 1, & \text{if } j = \arg\min_{k=1,\dots,K} \|f(\mathbf{x}_i) - \mathbf{c}_k\|^2, \\ 0, & \text{otherwise,} \end{cases} \tag{45}$$

and the centroids are then updated by

$$\mathbf{c}_k \leftarrow \mathbf{c}_k - \frac{1}{n_k^i}\big(\mathbf{c}_k - f(\mathbf{x}_i)\big)s_{k,i}, \tag{46}$$

where n_k^i denotes the count of samples assigned to cluster k before $\mathbf{x}_i$ been processed. Note that n_k^i acts as an adaptive learning rate.
In summary, DCN embeds the k-means term into the loss, uses explicit one-hot assignments, and performs mini-batch, alternating SGD updates: both the clustering term and network parameters are updated within each mini-batch. Consequently, the centroids are updated online rather than on the full dataset each epoch, enabling scalability to large datasets.
DC-$k$means [31] introduce auxiliary variables $\mathbf{y} = f_\theta(\mathbf{x})$ to decouple the representation learning and clustering objectives, enabling efficient optimization through Alternating Direction Method of Multipliers(ADMM). The loss function is defined as

$$\min_{\theta,\mathbf{Y},\mathbf{C}} \frac{1}{N}\sum_{i=1}^{N}\|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2 + \frac{\lambda}{2}\|\mathbf{y}_i - \mathbf{c}_i^*\|_2^2, \quad \text{s.t. } \mathbf{y}_i = f_\theta(\mathbf{x}_i), \tag{47}$$

where $\mathbf{c}_i^* = \arg\min_{\mathbf{c}_j}\|\mathbf{y}_i - \mathbf{c}_j\|_2^2$. And $\mathbf{x}_i$ and $\hat{\mathbf{x}}_i$ represent the original and reconstructed data points, respectively. With the augmented Lagrangian, each ADMM step has closed-form updates:

$$\mathbf{y}_i \leftarrow \frac{\lambda\,\mathbf{c}_i^* + \rho\big(f_{\theta_1}(\mathbf{x}_i) - \mathbf{u}_i\big)}{\lambda + \rho}, \tag{48}$$

$$\mathbf{u}_i \leftarrow \mathbf{u}_i + f_{\theta_1}(\mathbf{x}_i) - \mathbf{y}_i, \tag{49}$$

$$\mathbf{c}_j \leftarrow \frac{1}{N_j}\sum_{i\in\mathcal{C}_j}\mathbf{y}_i, \tag{50}$$

This method jointly optimizes reconstruction loss and clustering quality by learning representations, which also achieve a k-means friendly latent space. Specifically, they minimize a combined objective that balances autoencoder reconstruction error with the distance between learned representations and their assigned cluster centers. The ADMM formulation allows for closed-form updates of cluster assignments and centers, while network parameters are updated via standard gradient descent. As an instance of the Deep Cluster framework, DC-Kmeans has stronger scalability and can easily integrate different clustering algorithms (such as GMM) and network structures.

3.2 Probability Distribution Methods

A classic probabilistic model is Gaussian Mixture Model (GMM) [25], a type of model-Based methods. It assume that the dataset consists a series of probability distributions. Each cluster corresponds to a specific probability distribution and

the data points within a cluster are independently sample from the corresponding distribution. Clustering results are obtained by maximizing the likelihood function of the model parameters or through Bayesian inference. GMM assumes that data points are generated from a mixture of multiple Gaussian distributions. It can be described as following probability density function

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k), \tag{51}$$

where K is the number of Gaussian distributions, representing the number of mixture components; π_k is the weight of the k-th component, satisfying $0 \leq \pi_k \leq 1$ and $\sum_{k=1}^{K} \pi_k = 1$; $\mathcal{N}(x \mid \mu_k, \Sigma_k)$ represents the k-th Gaussian distribution, with μ_k as the mean vector and Σ_k as the covariance matrix. Each Gaussian distribution represents a distinct subpopulation within the dataset, corresponding to an individual cluster. GMM combines these components through a weighted sum, effectively deriving a probabilistic model that can represent complex datasets.

Deep Embedded Clustering(DEC) [34] augments autoencoder training with a probability-distribution–based clustering loss, aiming to use a deep network to map raw samples into a low-dimensional embedding space that is more conducive to clustering and to optimize the clustering objective directly within that space. After the pretraining stage of AE, DEC is followed by a fine-tuning stage. Firstly, the K-means algorithm is applied on the latent space generated by AE to obtain the initial cluster centers. In the next step, a soft assignment strategy based on the t-distribution is used to measure the similarity between each data point and all cluster centers, determining the probability q_{ij} of assigning a data point to a specific cluster. Specifically, the soft assignment probability q_{ij} represents a soft assignment, indicating the likelihood that the i-th data point belongs to the j-th cluster, and it is defined as

$$q_{ij} = \frac{(1 + \|z_i - \mu_j\|^2/\alpha)^{-\frac{\alpha+1}{2}}}{\sum_{j'}(1 + \|z_i - \mu_{j'}\|^2/\alpha)^{-\frac{\alpha+1}{2}}}, \tag{52}$$

where z_i is the latent representation of the i-th data point, corresponding to x_i after embedding, μ_j is the j-th cluster center, α is the degree of freedom of the Student's t-distribution. During each iteration of the training process, to harden the cluster assignments, DEC constructs a new target distribution p_{ij} based on the soft assignment probabilities q_{ij}. By optimizing the KL divergence between the soft assignment Q and the target distribution P, DEC optimizes the relationship between cluster centers and data points, causing data points to gradually converge to their most likely cluster centers. The construction of the target distribution is as:

$$p_{ij} = \frac{q_{ij}^2/f_j}{\sum_{j'} q_{ij'}^2/f_{j'}}, \tag{53}$$

where $f_j = \sum_i q_{ij}$ represents soft cluster frequencies. As a result, data points gradually converge toward their optimal cluster centroids. DEC combines an autoencoder with a self-training strategy to effectively cluster high-dimensional data, employing simultaneous deep clustering, which is one of the most widely used approaches. The representation module and the clustering module are

updated simultaneously through a single end-to-end framework. By optimizing this joint loss function, the discrepancy between the representation learning goals and the clustering goals is progressively minimized. Notably, DEC does not include a reconstruction loss during the clustering phase. However, this method still faces some challenges, such as the potential imbalance between optimization objectives of representation learning and clustering. Moreover, the model is very sensitive to initialization and easily to the cluster collapse, which means the model fails to from multiple distinct clusters and most of data points are assigned to only a few clusters. Due to this issue, the model requires constant adjustment of hyperparameters and loss function design.
The Improved Deep Embedded Clustering (IDEC) [6] extends the DEC framework by incorporating local structure preservation constraints into the representation learning process. IDEC enhances the representation capability by maintaining neighborhood structures from the original data space in the latent space, thus alleviating the problem of feature collapse observed in DEC.
The loss function of IDEC combines reconstruction loss and clustering loss, which is formulated as:

$$\mathcal{L} = \sum_{i=1}^{N}\sum_{j=1}^{K} q_{ij} \log \frac{q_{ij}}{p_{ij}} + \gamma \sum_{i=1}^{N} \|x_i - \hat{x}_i\|_2^2, \tag{54}$$

where x_i and $\hat{x}i$ represent the original and reconstructed data points, respectively. The term qij indicates the soft cluster assignment distribution, which is the similarity between embedded point z_i and cluster center μ_j measured by Student's t-distribution. While p_{ij} is the target distribution computed by squaring and normalizing q_{ij}. The parameter γ balances clustering and reconstruction objectives.
The key difference between IDEC and DEC is that IDEC maintains the reconstruction loss throughout training while DEC abandons it after pretraining. This joint optimization in IDEC preserves local data structure in the latent space, preventing the feature collapse problem common in DEC. Despite improved embedding quality, IDEC can still suffer from sensitivity to initialization and hyperparameters, especially the balance parameter γ, impacting its clustering stability and performance. IDEC does not always have better performance than DEC. For very simple or extremely complex datasets, the noisy local geometry may cause the reconstruction loss to override the clustering objective, thereby reduce overall performance.

3.3 SpectralNet

SpectralNet [28] also employs a deep neural network architecture to model the original data into a low-dimensional latent space, effectively facilitating traditional clustering algorithms. Traditional spectral clustering requires constructing a global similarity matrix and performing eigen-decomposition of the Laplacian matrix. The computational cost is $\mathcal{O}(n^3)$ and the storage cost is $\mathcal{O}(n^2)$. Furthermore, classical approaches does not accommodate out-of-sample extension (OOSE). SpectralNet addresses this issue by employing neural networks to learn mappings from input data to spectral embeddings, enabling rapid projection of unseen data points into the latent space without retraining the entire model.

SpectralNet first defines a symmetric similarity function $w : \mathbb{R}^d \times \mathbb{R}^d \to [0, \infty)$ to measure the similarity between data points. It then constructs an embedding function $y = f_\theta(x)$ implemented with neural networks that approximates the spectral mapping. Output $Z \in \mathbb{R}^k$ is passed through an orthogonality layer constructed via a Cholesky transform, yielding Y that approximates the eigenvectors in spectral clustering, whose columns satisfy the orthogonality constraint ($Y^\top Y = I$). The objective is to keep similar x_i and x_j are close in the latent space. Given w, the loss function is defined as

$$\mathcal{L}_{\text{SpectralNet}}(\theta) = \sum_{i,j} w(x_i, x_j) \, \|y_i - y_j\|^2 \tag{55}$$

$$= \operatorname{tr}(Y^\top L Y), \qquad \text{s.t. } Y^\top Y = I. \tag{56}$$

where $Y = [y_1, \ldots, y_n]^\top$ and $L = D - W$ is the graph Laplacian matrix. Optimization is performed via mini-batch stochastic gradient descent. During training process, mini-batches are sequentially fed into the network to produce embeddings in the latent space, followed by a k-means step to update cluster assignments. New samples can also be projected directly using the learned function f_θ, achieving efficient OOSE.
When constructing the affinity matrix W, a common choice of similarity measure assigns a positive weight to samples within a nearest-neighbor set and applies a Gaussian kernel with scale $\sigma > 0$, e.g.,

$$W_{ij} = \begin{cases} \exp\left(-\dfrac{\|x_i - x_j\|^2}{2\sigma^2}\right), & \text{if } x_j \text{ is one of the nearest neighbors of } x_i, \\ 0, & \text{otherwise.} \end{cases} \tag{57}$$

The matrix W is usually symmetrized as $W_{ij} \leftarrow (W_{ij} + W_{ji})/2$. Because Euclidean distance can be overly simplistic, SpectralNet employs a Siamese network [27] to estimate similarity. The Siamese network is trained with positive and negative pairs: positive pairs are formed from each point's nearest neighbors, whereas negative pairs are drawn from distant points. By minimizing the contrastive loss, the network learns an adaptive nearest-neighbor metric. This learned metric captures richer affinity relationships and significantly improves clustering quality.

4 Method

In this paper, we proposed a model that combines the representation capabilities of VAE and the clustering capabilities of NMF. This model employs a simultaneous update strategy to create an end-to-end framework. Basiri *et al.* [1] demonstrated that NMF achieve strong clustering performance on low-dimensional datasets. However, the real-world datasets are typically high-dimensional. To bridge this gap, we use VAE as representation module to extract compact representations from the raw inputs. The learned low-dimensional latent space is the used in NMF, which performs the clustering module.

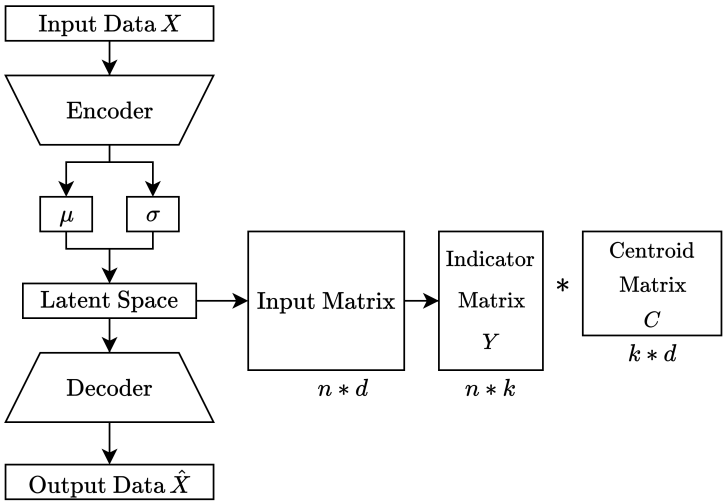


Figure 3: Proposed Model

4.1 Proposed Model

As described in Figure 3, the proposed architecture integrates the representation and clustering modules into a single end-to-end framework. Firstly, a VAE projects the raw input data X into a latent space, parameterized by the mean μ and variance σ. The resulting latent representation Encoder$(X) \in \mathbb{R}^{n\times d}$ serves as the input to the clustering module. Next, the clustering module use Semi-NMF to decompose Encoder(X) into an indicator matrix $Y \in \mathbb{R}_{+}^{n\times k}$ and a centroid matrix $C \in \mathbb{R}^{k\times d}$, where n is the number of samples, d the latent dimensionality, and k the number of clusters.

4.2 Objective Function

The loss of the proposed model is a weighted combination of the representation learning loss produced by the VAE and the clustering loss produced by the Semi-NMF. Eq. (42) gives the VAE evidence lower bound (ELBO), composed of a reconstruction term and a KL-divergence, while Eq. (9) defines the reconstruction error that drives clustering in NMF. We also add the PRIMP as a binary regularization term that drives the indicator matrix Y toward hard assignments Based on these three loss functions, we merge them into a single unified objective to enable end-to-end joint training, which is defined as follows

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{VAE}} + \lambda\,\mathcal{L}_{\text{NMF}} + R_{\text{PRIMP}}(Y) + \gamma\,\mathcal{L}_{\text{reg}}, \tag{58}$$

where

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] + \beta D_{\text{KL}}\big(q_\phi(z \mid x) \,\|\, p_\theta(z)\big), \tag{59}$$

is the representation-learning term; the coefficient β controls the impact of the KL penalty.
For the clustering term

$$\mathcal{L}_{\text{NMF}} = \|Z - YC\|^2, \qquad \text{s.t. } Y \geq 0. \tag{60}$$

where $\mathbf{Z} = \text{Encoder}(X) \in \mathbb{R}^{n \times d}$ is the latent representation. The objective of this term is to minimize the reconstruction loss of Semi-NMF while maintaining the non-negativity constraint on indicator matrix Y. The weighting factor $\lambda > 0$ controls the relative contribution of the two loss terms to balance the quality of latent space and clustering ability of the whole framework. A larger λ encourages the model to focus on the clustering performance, while a smaller λ emphasizes the reconstruction and regularization of the latent space to enhance the representation learning ability of model. For the binary regularization term, we adopt the PRIMP regularizer, which is defined as

$$R_{\text{PRIMP}}(y) = \begin{cases} \kappa(1 - |1 - 2y|), & \text{if } y \in [0, 1] \\ +\infty, & \text{otherwise} \end{cases} \tag{61}$$

$\kappa > 0$ controls the strength of the regularizer within the total loss. A larger κ can drive y closer to binary representation.
For the regularization term $\mathcal{L}_{\text{reg}}$, we employ the *Exclusive Lasso* regularization to enforce row-wise exclusivity. Mathematically, the penalty is defined as

$$\mathcal{L}_{\text{reg}}(Y) = \sum_{i=1}^{m} \Big(\sum_{j=1}^{n} |Y_{ij}| \Big)^2, \tag{62}$$

This squared $\ell_{1,2}$-norm penalizes multiple non-zero entries within the same row, thereby driving each row of the indicator matrix Y toward a one-hot representation. Because $\mathcal{L}_{\text{reg}}$ is convex and its proximal operator has a closed-form solution, it can be efficiently optimized within our proximal-gradient framework. By minimizing $\mathcal{L}_{\text{total}}$, gradients from the clustering loss are back-propagated to the VAE encoder, aligning the latent space with the Semi-NMF objective. Therefore, the model learns low-dimensional representations which are suited for the Semi-NMF clustering task and achieve clustering assignments in an end-to-end way.

4.3 Training Strategy

In this section, we present our detailed training strategy, including the VAE and Semi-NMF optimization processes along with a proximal gradient step designed for enforcing binary constraints. We first describe our alternating optimization method for jointly updating the parameters, followed by the application of a proximal gradient descent method. An overview of the complete training algorithm is summarized in Algorithm 1.

4.3.1 Alternating Optimization

We combine the VAE and Semi-NMF within a unified optimization framework, updating their parameters simultaneously. This joint update strategy enables gradient propagation from the clustering loss to the VAE, aligning the latent space with the NMF objective. As a result, the learned representation is influenced by the clustering objective to enhance latent space structuring more suitable for clustering.
Specifically, we employ an alternating optimization strategy to train our model, tightly coupling the learning process of Semi-NMF and VAE. Semi-NMF is

Algorithm 1: Training strategy of the VAE-NMF model

Input: Dataset X, number of clusters K, weighting λ, KL weight β
Output: indicator matrix Y
Initialize VAE parameters $\{\theta, \phi\}$; indicator matrix $Y \in \mathbb{R}_+^{n \times K}$; centroid matrix $C \in \mathbb{R}^{K \times d}$
while *not* *Converged* do
  $(\hat{X}, \mu, \log \sigma^2) \leftarrow \mathrm{VAE}(X)$
  $\mathcal{L}_{\mathrm{VAE}} \leftarrow \mathrm{Recon}(X, \hat{X}) + \beta\, \mathrm{KL}(\mu, \log \sigma^2)$
  $\mathcal{L}_{\mathrm{NMF}} \leftarrow \left\| z - YC \right\|_2^2$
  $\mathcal{L} \leftarrow \mathcal{L}_{\mathrm{VAE}} + \lambda\, \mathcal{L}_{\mathrm{NMF}}$
  Update (C, θ, ϕ) using $\mathcal{L}$ with optimizer $\mathcal{O}_C$
  Re-compute $(\hat{X}, \mu, \log \sigma^2)$ and losses $\mathcal{L}$
  Update (Y, θ, ϕ) using $\mathcal{L}$ with optimizer $\mathcal{O}_Y$
  $Y \leftarrow \mathrm{prox}_{\Delta}(Y)$
return (Y)

trained with an alternating optimization method where indicator matrix Y and centroid matrix C are updated separately. During each iteration, the first step is to optimize matrix C while keeping matrix Y fixed, then the second step is to optimize matrix Y while keeping matrix C fixed. In both sub-steps, the VAE parameters are updated simultaneously via gradients propagated from both the reconstruction loss and the clustering loss of Semi-NMF, effectively aligning latent representation learning with the clustering objective.
In our implementation, we employ two independent Adam optimizers: $\mathrm{optimizer}_Y$ and $\mathrm{optimizer}_C$ for updating matrix Y and C respectively. The $\mathrm{optimizer}_Y$ acts on both matrix Y and VAE parameters, while $\mathrm{optimizer}_C$ acts on both matrix C and VAE parameters. These two optimizers are alternately applied in each step to ensure balanced and stable updates across all model components.

4.3.2 Proximal Gradient Descent

Due to the non-convex and non-smooth nature of our binary regularizer, we append a proximal gradient step to every gradient update of the indicator matrix Y in addition to the alternating gradient updates above. After each alternating gradient update of the indicator matrix Y, we apply an element-wise proximal mapping with respect to the PRIMP regularizer, $Y^{t+1} = \mathrm{prox}_{R_{\mathrm{PRIMP}}}(Y)$, where Y is the intermediate result of the gradient step. The PRIMP regularizer admits the following four pieces closed-form proximal operator

$$\mathrm{prox}_R(y) = \begin{cases} 0, & y \le 2\kappa, \\ y - 2\kappa, & 2\kappa < y \le \frac{1}{2}, \\ y + 2\kappa, & \frac{1}{2} < y \le 1 - 2\kappa, \\ 1, & y \ge 1 - 2\kappa. \end{cases} \tag{63}$$

This piece-wise linear mapping reaches its minimum at the endpoints $y = 0$ and $y = 1$, while its maximum at the midpoint $y = \frac{1}{2}$. Thereby, this proximal operator driving each element of Y towards a relaxed one-hot representation.

The hyper-parameter $\kappa > 0$ scales the influence of the regularization term: A small κ is overly conservative, leading to inefficient gradient updates, whereas an overly large κ introduces optimization instability into the optimization process. In practice, we linearly increase κ during the training process, achieving a balance between optimization stability and binarization speed.
To further enhance the effectiveness and interpretability of the proximal operator in clustering tasks, we propose a dynamically adaptive proximal mapping strategy that enforces a strict one-hot constraint. This proximal descent method allows model to be effectively applied to exclusive hard clustering tasks. Specifically, we replace the threshold value $\frac{1}{2}$ in the PRIMP regularizer with the average value (denoted as *root*) of the largest and second-largest elements in each row of the indicator matrix. For each element y_i in a row:

- If $y_i < root$, it is pushed towards 0;
- If $y_i > root$, it is pushed towards 1;

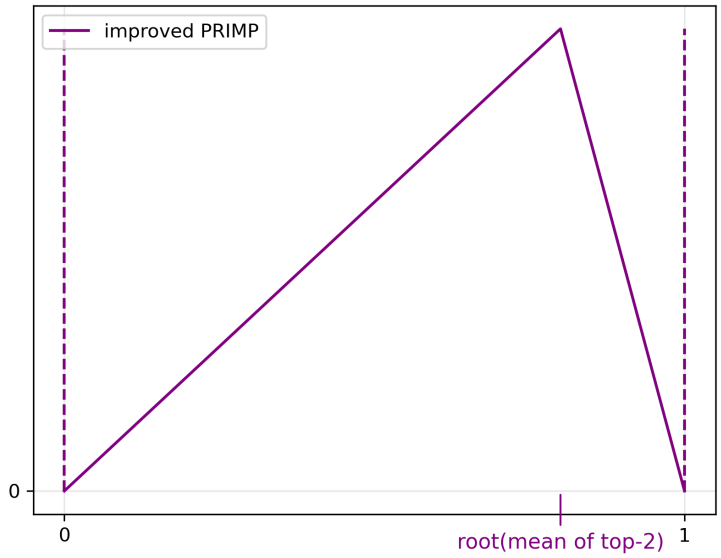


Figure 4: Improved PRIMP Regularization Functions

Figure 4 shows the regularization functions of the improved PRIMP. The update is defined as $y_i \leftarrow y_i - \kappa\,(root - y_i)$. The objective of this adjustment is to accelerate the convergence of elements smaller than *root* towards 0, and those larger than *root* towards 1. The step size of adjustment is determined by the distance from the element to *root*, elements closer to *root* are adjusted by a smaller amount. Through this update, we can speed up each row of the indicator matrix $\mathbf{Y}$ to approximate a one-hot vector.
We also apply the proximal operator for the Exclusive Lasso regularizer $\mathcal{L}_{\text{reg}}$, which is calculated by

$$Y^+ = \text{prox}_{\gamma\mathcal{L}_{\text{reg}}}(\tilde{Y}) = \arg\min_{Y} \frac{1}{2}\|Y - \tilde{Y}\|_F^2 + \gamma \sum_{i=1}^{m}\Big(\sum_{j=1}^{n}|Y_{ij}|\Big)^2, \qquad (64)$$

where $\tilde{Y}$ is the intermediate result and $\gamma > 0$ is the regularization strength for the Exclusive Lasso term. This proximal operator admits a closed-form solution that can be computed row-wise. For each row i, we have:

$$Y^+_{i,:} = \text{sign}(\tilde{Y}_{i,:}) \odot \max\left(|\tilde{Y}_{i,:}| - 2\gamma\|\tilde{Y}_{i,:}\|_1, 0\right), \tag{65}$$

where $\odot$ denotes element-wise multiplication. This method shrinks all elements within each row in proportion to ell_1 norm of that row, enhancing sparsity within the row dimension.
By combining the two regularization schemes, we drive the indicator matrix toward a strictly binary one-hot form in a complementary way: PRIMP enforces element-wise binarization, while Exclusive Lasso encourages a one-hot pattern across each row. By increasing the strength of PRIMP in the later stages of training, we can meet the strict one-hot constraint.

5 Experiments

5.1 Datasets

To investigate the performance and generalization ability of different algorithms, we evaluate our proposed method on one image dataset and one text dataset, and compare it with other methods.

5.1.1 MNIST

Image dataset is one of the most commonly used real-world datasets in deep clustering. We use the widely recognized MNIST dataset [13], which is a popular benchmark dataset for evaluation of clustering. MNIST contains 70,000 images of handwritten digits, including 60,000 training samples and 10,000 test samples. Each image is a 28×28 pixels grayscale image, where pixel values range from 0 to 255 and are usually normalized to $[0, 1]$. The dataset covers ten digit classes (0–9).

5.1.2 REUTERS-10k

The REUTERS dataset is a text dataset containing approximately 810,000 English news stories labeled with a category tree [16]. Following the procedure in DEC [34], we selected four root categories: corporate/industrial, government/social, markets, and economics as labels. After removing documents with multiple labels, we obtained 685,071 articles. Then we extract the TF-IDF features on the top 2,000 most frequent word stems and randomly sampled 10,000 examples to create the REUTERS-10K, due to some algorithms do not scale well to large-scale data.

5.2 Evaluation Metrics

To evaluate the proposed method, we employ several widely accepted metrics for unsupervised clustering tasks to exam the method from multiple perspectives. These metrics quantify the consistency between the clustering results and the true labels to compare the performance with baseline methods and competing methods. Specifically, we adopt the following five evaluation metrics:

5.2.1 Accuracy

Accuracy(ACC) is the most common metric for evaluating clustering performance. It represents the average proportion of clustered samples that are correctly classified and quantify the the overall correspondence between the clustering results and the ground-truth labels. Because clustering algorithms usually yield unlabeled outputs, the Hungarian algorithm [12] is typically employed before computing ACC to determine the optimal mapping between the cluster labels and the true labels. ACC is calculated as follows

$$\text{ACC} = \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(l_i = \text{map}(c_i)) \tag{66}$$

where $\text{map}(\cdot)$ denotes the optimal mapping function obtained via the Hungarian algorithm, l_i is the ground-truth label of the i-th sample; c_i is the cluster label assigned to the i-th sample; N is the total number of samples;$\mathbf{1}(\cdot)$ is the indicator function.

5.2.2 Normalized Mutual Information

Normalized Mutual Information (NMI) is another common metric for evaluating clustering performance. It quantifies the information correlation between the clustering results and the ground-truth labels. Its value ranges from $[0, 1]$ where the higher value indicates the better clustering performance. NMI is calculated as follows

$$\text{NMI} = \frac{2 \cdot I(L;C)}{H(L) + H(C)} \tag{67}$$

where $I(L;C)$ is the mutual information between the ground-truth labels L and the cluster assignments C, which is defined as

$$I(L;C) = \sum_{i,j} P(l_i, c_j) \log \frac{P(l_i, c_j)}{P(l_i)P(c_j)} \tag{68}$$

where $P(l_i, c_j) = \frac{|l_i \cap c_j|}{N}$ is the joint probability of a sample belonging to both ground-truth class l_i and cluster c_j, $P(l_i) = \frac{|l_i|}{N}$ and $P(c_j) = \frac{|c_j|}{N}$ are the marginal probabilities. $H(L)$ and $H(C)$ are the entropies of the ground-truth labels and cluster assignments, respectively:

$$H(L) = -\sum_{i} P(l_i) \log P(l_i), \quad H(C) = -\sum_{j} P(c_j) \log P(c_j) \tag{69}$$

5.2.3 Adjusted Rand Index

Rand Index (RI) is also a metric for evaluating clustering algorithm performance, quantifying the consistency by comparing the assignment relationships of all sample pairs. RI is calculated as follows

$$\text{RI} = \frac{TP + TN}{\binom{N}{2}} \tag{70}$$

where TP denotes the number of true positive pairs and TN denotes the number of true negative pairs; $\binom{N}{2}$ is the total number of sample pairs. However,

considering that RI is not close to 0 for random clustering, it is easy to get high scores but meaningless results, which is difficult to interpret intuitively. The Adjusted Rand Index (ARI) corrects for this issue

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]} \tag{71}$$

Through this adjustment calculation, ARI better reflects the actual performance of clustering. Its value ranges from $[-1, 1]$, where higher value indicates better clustering performance.

5.2.4 Purity

Purity is an intuitive metric for evaluating clustering algorithm performance. It reflects the class consistency within each cluster by measuring the proportion of the dominant class in each cluster. Purity is calculated as follows

$$\text{Purity} = \frac{1}{n} \sum_{i=1}^{k} \max_{j} |C_i \cap L_j|, \tag{72}$$

Purity ranges from $[0, 1]$ where higher value indicates greater cluster homogeneity and thus better clustering performance. A shortcoming of Purity is its sensitivity to the number of clusters, when the number of clusters equals the number of samples, Purity reaches 1.

5.2.5 F1_measure

The F1 score is the harmonic mean of *precision* and *recall*, balancing false positives and false negatives in a single metric. For class j:

$$\text{Precision}_j = \frac{\text{TP}_j}{\text{TP}_j + \text{FP}_j}, \qquad \text{Recall}_j = \frac{\text{TP}_j}{\text{TP}_j + \text{FN}_j}, \tag{73}$$

$$F1_j = \frac{2\,\text{Precision}_j\,\text{Recall}_j}{\text{Precision}_j + \text{Recall}_j}. \tag{74}$$

In multi-class settings, the *weighted* F1 averages per-class scores by their support:

$$F1_{\text{weighted}} = \sum_{j=1}^{k} \frac{n_j}{n} F1_j, \tag{75}$$

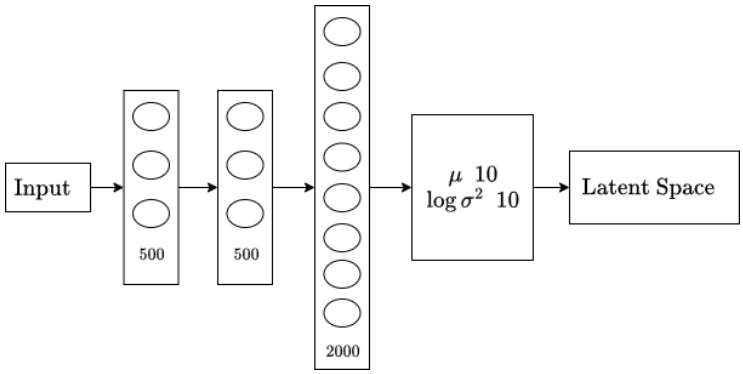
where n_j is the number of samples in class j and n is the total sample count. The F1 score ranges from 0 to 1; higher values indicate a better balance between precision and recall.

5.3 Implementation

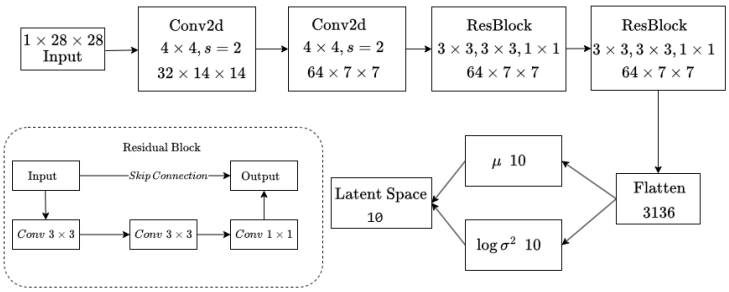
We first present the model architectures, showing how the VAE and the matrix-factorization module are build. Then, we describe the optimizer settings in training procedure. Experiments are performed on a single NVIDIA RTX 3070Ti (8GB) and an Intel Core 10850K CPU. Complete training on MNIST takes 70 minutes.

5.3.1 Model Architectures

We implement a deep VAE following the same dimensional configuration as DEC[34] for easier comparison. The network adopts a symmetric encoder–decoder structure with the configuration `D-500-500-2000-10`, where D represents the input feature dimension and 10 denotes the latent dimension. All layers in the network are fully connected.
We also implement another convolutional VAE following the configuration as VQ-VAE [33]. This architectures employs convolutional layer with residual connections for stable training and effective feature learning. Specifically, the encoder progressively reduces spatial dimensions through two convolutional layers while expanding feature channels, followed by two residual blocks that maintain spatial resolution while learning complex representations. Each residual block contains three convolutional layers with skip connections that add the input directly to the output. This architectures is also symmetric as the decoder mirrors this structure in reverse. As shown in Figure 5, the encoder structures of the two VAE variants are depicted.



(a) Encoder of the Fully Connected VAE



(b) Encoder of the Convolutional VAE

Figure 5: Side-by-side comparison of the encoder components of two VAE architectures: (a) and (b). Since both models are fully symmetric, only the encoder parts are illustrated.

To prevent overfitting, dropout with a rate of 0.2 is applied to all hidden layers in the encoder. The encoder's output layer does not use dropout to preserve the integrity of latent–parameter estimation. In this task, we focus feature-learning capability of the VAE, so the decoder layers do not employ dropout. During

Algorithm 2: K-means++ Initialization

Input: Dataset $X \in \mathbb{R}^{n \times d}$, number of clusters K
Output: Initial cluster centers $C \in \mathbb{R}^{K \times d}$
Initialize random number generator
$C \leftarrow \emptyset$ **for** $k = 1$ ***to*** K **do**
  if $k = 1$ **then**
    $idx \leftarrow \text{RandomChoice}(n)$
  else
    $D \leftarrow \min_{c \in C} \|X - c\|_2^2$
    $p \leftarrow D / \sum D$
    $idx \leftarrow \text{RandomChoice}(n, p)$
  $C \leftarrow C \cup \{X[idx]\}$
return C

initialization, all linear layers are initialized using Xavier uniform initialization, where the gain is calculated specifically for ReLU activations, and all bias terms are initialized to zero.

For the Matrix-factorization clustering model, the dimension of the indicator matrix is $N \times K$, where N represents the number of samples and K denotes the number of clusters. The central matrix $C \in \mathbb{R}^{k \times d}$, where d is the dimension of the VAE latent layer. During initialization, we employ a uniform distribution. In particular, we adopt the *k-means++* initialization strategy for C, which selects initial centers that are more dispersed, resulting in broader initial coverage of the feature space. One challenge of matrix factorization is the strong coupling between matrices Y and C, which can easily trap in local minima during alternating updates. An effective initialization strategy can solve this issue by decoupling Y and C to some extent, thus helping model converge to the global optimum. A detailed description of the *k-means++* initialization strategy is presented in Algorithm 2.

5.3.2 Hyperparameters

In Section 4.2, we introduce four hyperparameters associated with different regularization terms in the loss function: β, λ, κ, and γ. Specifically, β controls the strength of KL regularization in the VAE; λ controls the strength of the clustering loss derived from matrix factorization; κ controls the strength of the PRIMP binary regularization; and γ controls the strength of the Exclusive-Lasso regularization.

For the KL regularization term, inspired by the warm-up strategy proposed in Ladder [29], we gradually increase its strength as training progresses, scaling from an initial value of 0.1 up to 1. Similarly, for the hyperparameter λ, we adopt a warm-up strategy. Its initial value is set to 0 during the VAE pretraining stage. After completion of pretraining, we gradually increase λ from an initial value of 0.1 up to 1. The hyperparameter κ is activated when the clustering loss join. Its value progressively increases throughout training, starting from 1×10^{-4} and doubling every 50 epochs. The hyperparameter γ follows the same strategy as κ, initially set to 1×10^{-3} and doubling every 50 epochs.

6 Results and Discussion

6.1 Comparison to Baseline Methods

Table 1: Clustering performance on **MNIST** (ACC, NMI, ARI, F1, and Purity) compared across different methods *(best in each column shown in **bold**).*

Method	**ACC**	**NMI**	**ARI**	**F1**	**Purity**
k-means	0.5229	0.4854	0.3498	0.5282	0.5741
NMF	0.4511	0.3862	0.2747	0.4448	0.4586
NMF(k-means++ Init)	0.4627	0.4197	0.2876	0.4572	0.5152
AE+k-means	0.6432	0.6362	0.5283	0.6354	0.6913
AE+NMF	0.6359	0.6543	0.5781	0.7318	0.7359
VAE+k-means	0.7389	0.6529	0.5764	0.7414	0.7389
VAE+NMF	0.6674	0.6343	0.5527	0.6495	0.7160
GMM	0.3691	0.3074	0.1798	0.3768	0.4047
DEC	0.7713	0.7441	0.6740	0.7602	0.8129
DC+k-means	0.8015	0.7448	0.6890	0.7900	0.8015
DC+GMM	0.8555	0.8318	0.7823	0.8200	0.8555
SpectralNet(Euclidean)	0.8150	**0.8432**	0.7723	0.7943	**0.8680**
DCN	0.8190	0.8042	0.6561	0.7207	0.7876
VAE-NMF (Fully Connected)	0.8326	0.7115	0.6776	0.8331	0.8326
VAE-NMF (Convolutional)	**0.8568**	0.8412	**0.7933**	**0.8668**	**0.8654**

Table 1 shows a comparison of clustering performance between our proposed VAE-NMF method and various baseline methods on MNIST dataset. These methods are compared across five standard evaluation metrics: ACC, NMI, ARI, F1-score, and Purity. Our proposed VAE-NMF method achieves competitive results, outperforming most distance-based baselines and approaching the performance of state-of-the-art probabilistic models. Table 2 shows the same evaluation metrics of these methods on Reuters-10k dataset and Appendix C shows the results on Fashion MNIST dataset.

In Section 2.1.3, we established the theoretical equivalence of k-means and NMF for clustering tasks, which aligns well with our experimental results. NMF and K-means achieved similar performance on the original dataset and the latent spaces derived through feature learning, as demonstrated by ACC values of 0.4511 and 0.5229, respectively. The minor performance differences observed are likely attributable to variations in initialization and optimization procedures. These small differences support our theoretical analysis, confirming the applicability of NMF in clustering-related tasks.

Methods combining deep feature learning with clustering consistently outperform original clustering methods by a significant advantage. Representations learned from AE and VAE provide discriminative features useful for clustering algorithms. For instance, the VAE+k-means achieves an ACC of 0.7389, significantly outperforming traditional methods. The hierarchical feature extraction capability of deep networks effectively captures complex patterns, which linear methods such as NMF or K-means struggle to catch. Methods that typically perform poorly on high-dimensional raw data can exhibit substantially improved clustering performance in low-dimensional latent spaces.

Table 2: Clustering performance on **Reuters-10k** (ACC, NMI, ARI, F1, and Purity) compared across different methods *(best in each column shown in **bold**)*.

Method	**ACC**	**NMI**	**ARI**	**F1**	**Purity**
k-means	0.6986	0.4118	0.3691	0.6897	0.7523
NMF	0.6379	0.3710	0.2988	0.6086	0.6847
NMF(k-means++ Init)	0.7154	0.5245	0.4782	0.7277	0.8243
AE+k-means	0.7489	0.4413	0.3372	0.7350	0.8421
AE+NMF	0.7368	0.4575	0.3456	0.7524	0.7718
VAE+k-means	0.7436	0.4472	0.3675	0.7343	0.8061
VAE+NMF	0.7401	0.4241	0.3290	0.7426	0.7880
GMM	0.6603	0.4202	0.3351	0.6676	0.7523
DEC	0.6974	0.4495	0.4676	0.7234	0.7616
DC+k-means	0.6404	0.5128	0.3775	0.5578	0.7309
DC+GMM	0.6612	0.4225	0.3328	0.5217	0.7012
SpectralNet(Euclidean)	0.7643	0.5189	0.5034	0.7401	0.7948
DCN	0.7194	0.4683	0.5065	0.7187	0.7989
VAE-NMF (Fully Connected)	0.7508	0.4926	0.4965	0.7577	0.8136
VAE-NMF (Convolutional)	0.4301	0.1238	0.3769	0.2622	0.4315

The proposed fully connected VAE-NMF method significantly outperforms AE-based approaches in all evaluated metrics, with an ACC of 0.8326 compared to 0.6432 and 0.6359, respectively. Incorporating the matrix factorization-based clustering loss greatly enhances the feature-learning capabilities of vAE enables end-to-end unified optimization, simultaneously optimizing the objectives for VAE and NMF, thereby generating inherently clustering-friendly representations. This joint optimization procedure clearly achieve the better performance than obtained by using VAE and NMF separately in two stages, underscoring the importance of unified optimization in deep clustering. Compared with DCN, which employs k-means for clustering loss computation, our method achieves comparable results, such as ACC (0.8326 vs. 0.8190), ARI (0.6776 vs. 0.6561), F1-score (0.8331 vs. 0.7207), and Purity (0.8326 vs. 0.7876). Although both approaches use Euclidean distance for cluster loss computation, DCN requires manual updates for clustering loss, thereby lacking a fully end-to-end optimization capability.

6.2 Loss Curve Analysis

Figure 6 shows the evolution of five evaluation metrics throughout the training process, while Figure 7 shows the trajectories of the total loss, reconstruction loss, KL divergence, and clustering loss. During the initial 100 epochs, we set the clustering loss weight to zero and gradually warm up the KL divergence weight, focusing primarily on VAE pre-training. This strategy results in a sharp decline in the reconstruction loss, enabling the model to learn a simple but well-separated latent representation.

After 100 epochs, we activate the clustering loss and begin computing matrix factorization to derive the clustering objective. At the same time, we calculate evaluation metrics based on the matrix factorization results. Consequently,

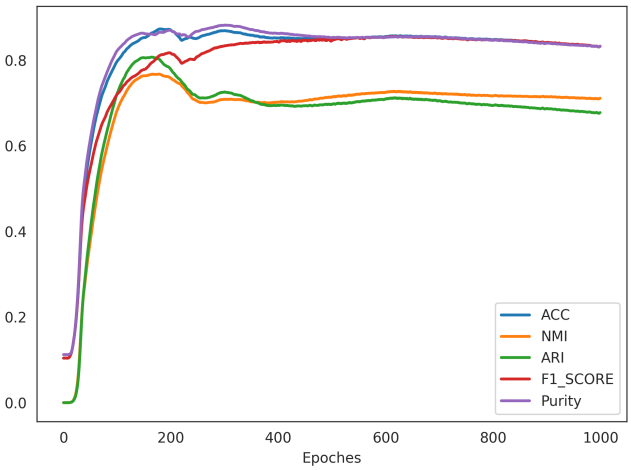


Figure 6: Evolution of Clustering Performance Metrics over Training Epochs.

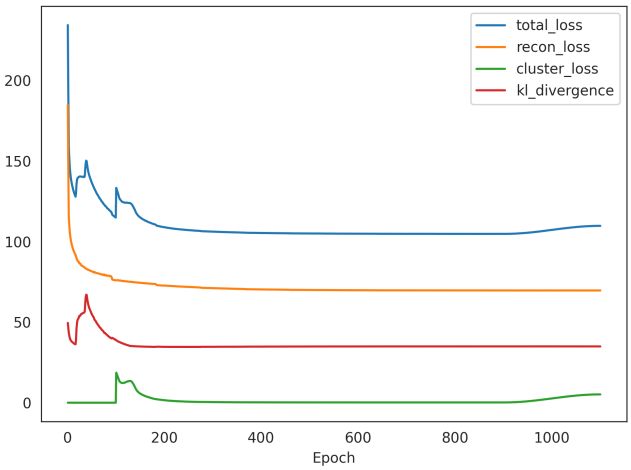


Figure 7: Trends of Total Loss and Its Components over Training Epochs.

epoch 0 in Figure 6 corresponds to epoch 100 in Figure 7. The introduction of the matrix factorization loss shows a peak, accompanied by a slight increase in the total loss. Subsequently, the performance metrics show a fast and sustained improvement.
Throughout the training process, we progressively increase the weight of the proximal operator to encourage the indicator matrix towards one-hot embeddings. This strategy leads to an increase in the clustering loss during the final training phase, as the model enforces harder cluster assignments.

6.3 Latent Space Visualization

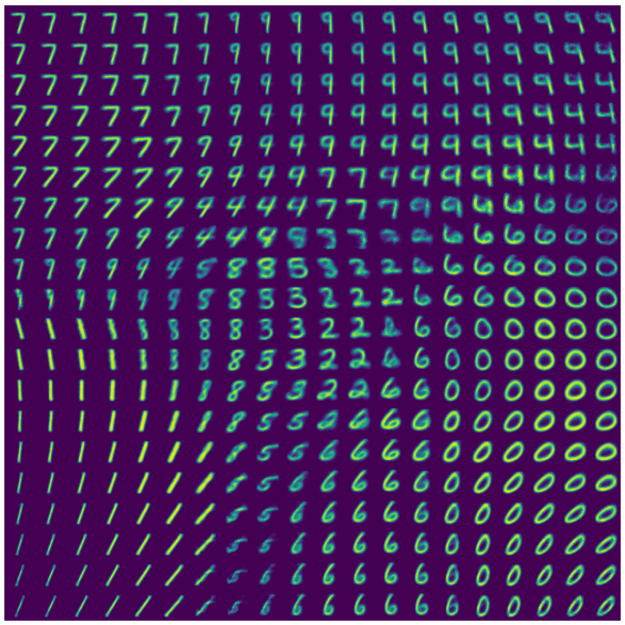


Figure 8: Manifold visualization: Generated Digits from Latent Representations.

The figure 8 shows the distribution of generated samples obtained through uniform sampling in a two-dimensional latent space. Our proposed method is based on a VAE framework, assuming the latent space follows a Gaussian distribution. We uniformly sample coordinates from a square-shaped latent space, reconstruct the corresponding data samples using the decoder, and visualize these reconstructed images at their respective coordinates.
The visualized grid images exhibit a smooth and continuous latent space. Digits generated from neighboring points in the latent space tend to be similar, and positions of visually similar digits, such as 7 and 9, 6 and 0, are adjacent to each other. Additionally, the generated samples from neighboring position show no significant changes in appearance, indicating that the model effectively learns meaningful representations without explicit supervised conditions. This continuous embedding representation accurately captures structural features in the data, facilitating clustering.

Furthermore, the visualization clearly reveals distinct digit clusters. Within the latent space, each digit occupies a close region with a smooth boundary, indicating the model's ability to effectively separate clusters in the latent space. Transition areas are not chaotic or random, which helps clustering algorithms achieve more precise and stable outcomes. Each digit class includes diverse styles, such as varying tilt angles, thickness, handwriting styles, and shapes, which preserving meaningful class differences in the learned representations. Overall, these results validate the effectiveness of our approach in clustered tasks within the latent space.

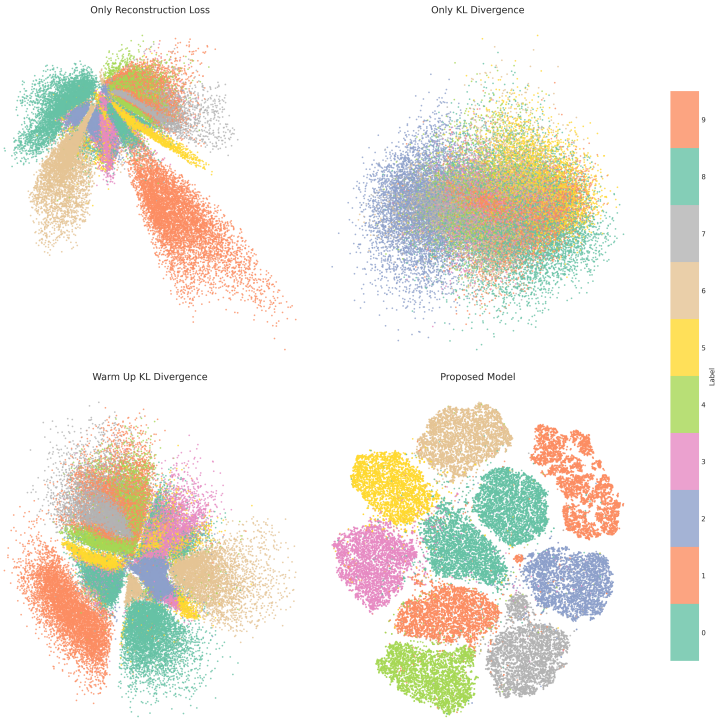


Figure 9: Latent Space Visualization

The figure 9 shows the latent space learned by the VAE under different loss function.

When using only reconstruction loss, the latent space shows clear structural features. Data points from the same class form tight clusters, but the overall distribution appears as an irregular manifold. This representation preserves the original topology structure of the data, but leads to significant overlap between different classes, unclear boundaries, and uneven utilization of the latent space. Using only KL divergence excessively constrains the latent space into a standard normal distribution, but resulting in complete mixing of representations from all classes. This over-regularization severely reduces class-specific information. Although the prior distribution assumption is satisfied, the structural details of the data are lost.

By implementing a warm-up strategy for KL divergence, a balance between reconstruction and regularization is achieved. Some clustering characteristics

are preserved, resulting in a more organized latent space. Different classes start to separate, though some overlap remains.
With the introduction of clustering loss in proposed model, the latent space demonstrates better clustering properties. Samples from the same class form very close and compact, clusters with clear boundaries separating different classes. The latent space is uniformly utilized, and the clusters exhibit more regular shapes. Explicit clustering loss guides the latent representation to directly optimize clustering objectives, enabling even simple clustering algorithms to achieve excellent performance.

6.4 Confusion Map Insights

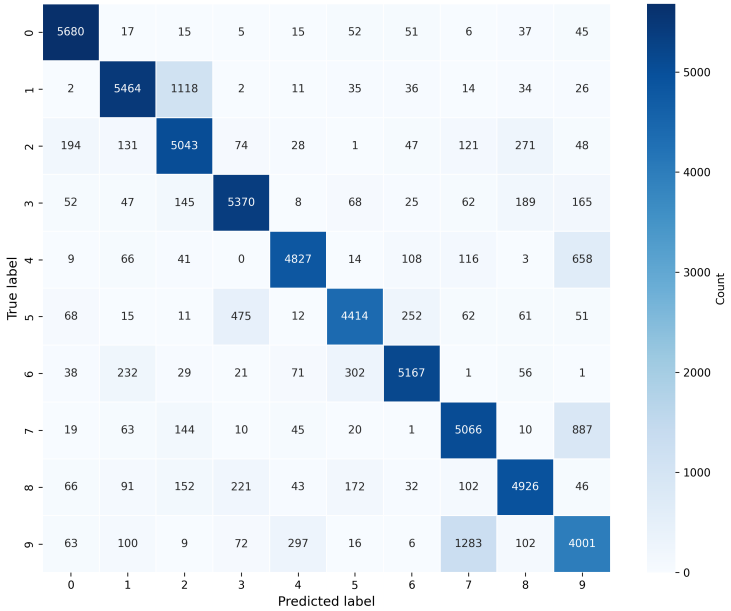


Figure 10: Confusion Map

The figure 10 shows confusion matrix between clustering results and ground truth labels, where the x-axis represents predicted labels and the y-axis represents true labels. Color intensity indicates the number of samples, with darker colors corresponding to higher counts. The diagonal elements of the confusion matrix are significantly stronger than the off-diagonal elements, demonstrating that our method achieves good clustering performance.
The confusion matrix primarily reveals concentrated error patterns, mainly occurring between digits 4 and 9, 7 and 9, 4 and 9, and 1 and 2. These digit pairs share some highly similar structural features. Digits 0, 3, and 6 show relatively good clustering results without obvious confusion patterns.
The confusion map demonstrates the interpretability of our method. The main confusion patterns are concentrated among visually similar digits. These similar digits may remain similar in the latent space, and the NMF have difficulty distinguishing the resulting non-convex or overlapping regions. In future work,

contrastive loss can be applied to highly confused combinations, or hierarchical clustering can be used to first separate error-prone digits, such as digit 7.

6.5 Impact of Cluster Number k

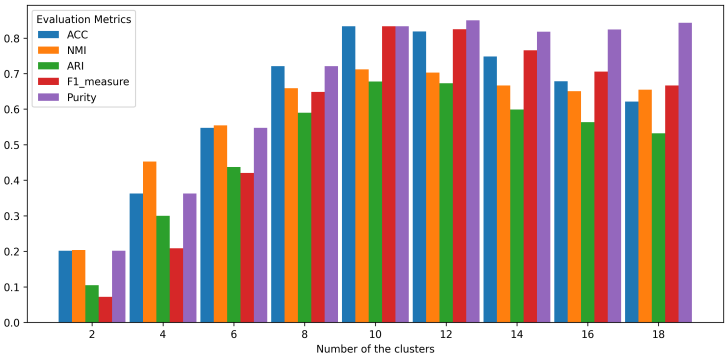


Figure 11: Clustering performance measured by ACC, NMI, ARI, F1-measure, and Purity for varying number of clusters k.

Figure 11 shows the clustering performance under various evaluation metrics when the number of clusters k ranges from 2 to 18. As shown in the figure, clustering performance improves as the number of clusters increases from 2 to around 10. Metrics such as ACC, NMI, ARI, and F1 measure consistently rise and achieve their peak performance near $k = 10$, indicating the optimal clustering result at this point. Purity exhibits a similar pattern, reaching a high value at $k = 10$ and having a slight fluctuation afterward.

When the number of clusters exceeds 10, improvements in most metrics stabilize or marginally decline. Specifically, ARI and NMI values slightly decrease after reaching their peaks, suggesting that an much increase in the number of clusters might result in fragmentation or less meaningful groupings. In contrast, purity maintains relatively high values despite an increase in clusters, indicating its tendency to favor homogeneous clusters at the cost of possibly over-segmenting the data.

Considering that clustering algorithms typically require specifying the number of clusters beforehand, this practice is not entirely unsupervised since it implicitly provides additional information. Ideally, fully unsupervised methods should autonomously uncover the inherent structure of the data without prior explicit information regarding the number of categories or clusters. The significant improvement observed across various metrics when the number of clusters approaches the true number of underlying groups demonstrates that a suitable choice of k should reflect the intrinsic distribution characteristics of the data.

These observations show the importance of carefully selecting the number of clusters. Choosing an appropriate value for k (around 10 in this case) is crucial to balancing clustering accuracy and interpretability.

Table 3: Clustering performance (**ACC**, **NMI**, **ARI**, **F1_measure**, and **Purity**) of fully connected and convolutional architectures on MNIST and Reuters-10K datasets.

Dataset	Architecture	ACC	NMI	ARI	F1_measure	Purity
MNIST	Fully Connected	0.8326	0.7115	0.6776	0.8331	0.8326
	Convolutional	**0.8568**	**0.8412**	**0.7933**	**0.8668**	**0.8654**
Reuters-10K	Fully Connected	**0.7508**	**0.4926**	0.4965	**0.7577**	**0.8136**
	Convolutional	0.4301	0.1238	**0.3769**	0.2622	0.4315

6.6 Performance on Different VAE Structure

The experimental results presented in Table 3 demonstrate the performance two VAE architectures illustrated in Section 5.3.1: fully connected and convolutional. We test these two architectures across two distinct datasets: MNIST (image-based) and Reuters-10K (text-based) as described in Section 5.1.

On the MNIST dataset, the convolutional VAE architecture significantly outperforms fully connected architecture across all clustering metrics, achieving an accuracy of 0.8554 compared to 0.8326 for the fully connected model. This improvement is particularly noticeable in terms of NMI and ARI, where the convolutional model scores 0.8412 and 0.7933 respectively, compared to 0.7115 and 0.6776 for the fully connected model. These results show the effectiveness of convolutional structures in capturing inherent structure in image data, demonstrating better representational capabilities that improving clustering performance.

Conversely, on the Reuters-10K text dataset, the fully connected architecture achieves better performance than the convolutional architecture. The fully connected model achieves an accuracy of 0.6434 and a purity of 0.7112, where the convolutional model achieves an accuracy of 0.4301 and purity of 0.4315. The textual data structure, characterized by sparse and high-dimensional representations, appears to be more effectively captured by fully connected layers, which better handle non-spatial relation and features in text embeddings.

This observed difference in performance highlights the importance of aligning neural network architectures with the intrinsic properties of the data. Convolutional structures are particularly good at extracting meaningful patterns from structured image data. In contrast, fully connected architectures demonstrate greater flexibility and efficacy in modeling high-dimensional, sparse textual representations. Overall, these findings suggest that the choice of VAE architecture should be carefully considered based on data modality. Future research could explore hybrid architectures or adaptive models that dynamically combine these architectural features to optimize performance across diverse data types.

6.7 Effect of Exclusive Lasso

In our loss function we add an Exclusive Lasso term to promote competition within each group, pushing every row of the indicator matrix toward a one-hot embedding. To measure the effect of Exclusive Lasso and show whether it truly induces exclusivity inside each indicator vector, we evaluate it with the Group

Dominance Ratio (GDR), defined as

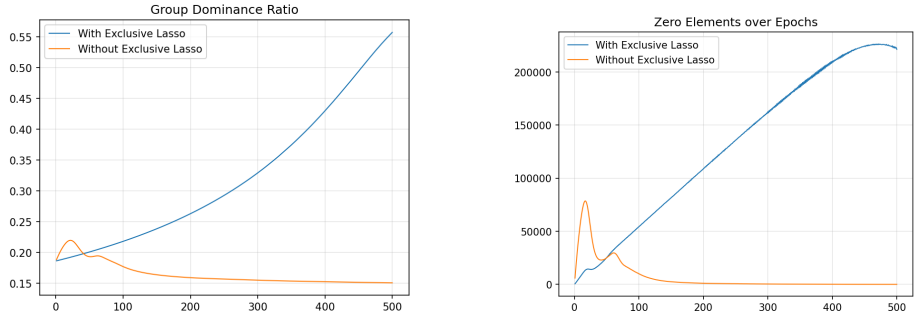
$$C_i = \frac{\max_j |y_{ij}|}{\sum_j |y_{ij}|}, \tag{76}$$

where y_{ij} is the j-th element in group i which is the i-th row of indicator matrix. If the denominator is zero, we set $C_i = 0$. For each row of the indicator matrix we obtain an GDR, and then calculate an overall score by averaging the GDR across all groups:

$$\bar{C} = \frac{1}{N} \sum_{i=1}^{N} C_i. \tag{77}$$

Figure 12 illustrates the training dynamics of the Group Dominance Ratio (GDR) and the number of zero elements, with and without the Exclusive Lasso regularizer. Because the improved PRIMP–based proximal operator would forcibly drive each row of the indicator matrix towards a one–hot vector, we disable it here to isolate the effect of the Exclusive Lasso within the matrix factorization. With the Exclusive Lasso in, the number of zero entries grows almost linearly across epochs. Whereas the count quickly decays to nearly zero without Exclusive Lasso. The mean GDR measures row exclusivity, showing a similarly divergent behavior: it rises monotonically under the Exclusive Lasso, confirming the original design intention of promoting competition within the group.

These two trends together indicate that the Exclusive Lasso simultaneously increases the exclusivity of the indicator vectors and induces substantial sparsity, leading to more interpretable clustering results.



(a) (b)

Figure 12: The (a) evaluates the mean GDR of indicator matrix in each epoch. (b) shows the number of zero elements in the indicator in each epoch

7 Conclusion

Recent studies have validated the effectiveness of coupling AE–based representation learning module with k-means across different domains. However, the

classical k-means objective depends on discrete assignment variables, preventing direct optimization by gradient descent in an end-to-end model.

To overcome this limitation, we propose an end-to-end clustering framework that fuses VAE-based representation learning module with a NMF formulation that is provably equivalent to k-means. Our method aims to map high-dimensional data to a clustering-friendly latent space in which similar samples are compactly distributed in latent space. Specifically, we relax the orthogonality constraint on the indicator matrix and defined the k-means loss as a differentiable NMF objective. By employing alternating optimization and proximal gradient descent within a single loss function, the model simultaneously updates latent representations and cluster assignments, which can also make sure the cluster assignments is sparse and interpretable. This joint training strikes a balance between representation fidelity and clustering accuracy, enabling mini-batch updates and real-time visualization of clustering results.

Experiments on the MNIST, Fashion-MNIST, and Reuters-10K datasets demonstrate that the proposed model substantially surpasses traditional k-means, NMF, and a wide range of deep clustering baselines in terms of ACC, NMI, ARI, F1 score, and Purity, while scaling gracefully to large, high-dimensional datasets. These results not only confirm the theory that NMF is equivalent to k-means, but also show its power when embedded in a VAE.

Future work can explore more flexible generative priors, such as Gaussian-Mixture [9] or VampPrior [32], to replace the isotropic standard Gaussian $\mathcal{N}(0, 1)$ which is typically assumed in VAEs—a simplification that is often too rigid for data with multiple clusters or complex manifolds. In addition, incorporating contrastive losses between similar classes may further enhance both performance and interpretability.

References

[1] Fahimeh Basiri et al. "A Comparative Study of K-means and NMF Clustering Algorithms". In: *2023 2nd International Engineering Conference on Electrical, Energy, and Artificial Intelligence (EICEEAI)*. IEEE. 2023, pp. 1–4.

[2] Mathilde Caron et al. "Deep clustering for unsupervised learning of visual features". In: *Proceedings of the European conference on computer vision (ECCV)*. 2018, pp. 132–149.

[3] Chris Ding, Xiaofeng He, and Horst D Simon. "On the equivalence of nonnegative matrix factorization and spectral clustering". In: *Proceedings of the 2005 SIAM international conference on data mining*. SIAM. 2005, pp. 606–610.

[4] Chris H.Q. Ding, Tao Li, and Michael I. Jordan. "Convex and Semi-Nonnegative Matrix Factorizations". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 32.1 (2010), pp. 45–55.

[5] Jiangzhang Gan et al. "Non-negative matrix factorization: a survey". In: *The Computer Journal* 64.7 (2021), pp. 1080–1092.

[6] Xifeng Guo et al. "Improved deep embedded clustering with local structure preservation." In: *Ijcai*. Vol. 17. 2017, pp. 1753–1759.

[7] Sibylle Hess, Katharina Morik, and Nico Piatkowski. "The PRIMPING routine—Tiling through proximal alternating linearized minimization". In: *Data Mining and Knowledge Discovery* 31 (2017), pp. 1090–1131.

[8] Anil K Jain, M Narasimha Murty, and Patrick J Flynn. "Data clustering: a review". In: *ACM computing surveys (CSUR)* 31.3 (1999), pp. 264–323.

[9] Zhuxi Jiang et al. "Variational deep embedding: An unsupervised and generative approach to clustering". In: *arXiv preprint arXiv:1611.05148* (2016).

[10] Jingu Kim and Haesun Park. *Sparse nonnegative matrix factorization for clustering*. Tech. rep. Georgia Institute of Technology, 2008.

[11] Diederik P Kingma. "Auto-encoding variational bayes". In: *arXiv preprint arXiv:1312.6114* (2013).

[12] Harold W Kuhn. "The Hungarian method for the assignment problem". In: *Naval research logistics quarterly* 2.1-2 (1955), pp. 83–97.

[13] Yann LeCun et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.

[14] Daniel Lee and H Sebastian Seung. "Algorithms for non-negative matrix factorization". In: *Advances in neural information processing systems* 13 (2000).

[15] Daniel D Lee and H Sebastian Seung. "Learning the parts of objects by non-negative matrix factorization". In: *nature* 401.6755 (1999), pp. 788–791.

[16] David D Lewis et al. "Rcv1: A new benchmark collection for text categorization research". In: *Journal of machine learning research* 5.Apr (2004), pp. 361–397.

[17] Tao Li and Chris Ding. "The relationships among various nonnegative matrix factorization methods for clustering". In: *Sixth International Conference on Data Mining (ICDM'06)*. IEEE. 2006, pp. 362–371.

[18] Tao Li and Sheng Ma. "IFD: iterative feature and data clustering". In: *Proceedings of the 2004 SIAM International Conference on Data Mining*. SIAM. 2004, pp. 472–476.

[19] Zhihui Li et al. "Balanced clustering via exclusive lasso: A pragmatic approach". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018.

[20] Chih-Jen Lin. "Projected gradient methods for nonnegative matrix factorization". In: *Neural computation* 19.10 (2007), pp. 2756–2779.

[21] Stuart Lloyd. "Least squares quantization in PCM". In: *IEEE transactions on information theory* 28.2 (1982), pp. 129–137.

[22] Erxue Min et al. "A survey of clustering with deep learning: From the perspective of network architecture". In: *IEEE Access* 6 (2018), pp. 39501–39514.

[23] Shakir Mohamed et al. *Monte Carlo Gradient Estimation in Machine Learning*. 2020. arXiv: `1906.10652 [stat.ML]`. URL: `https://arxiv.org/abs/1906.10652`.

[24] Nairouz Mrabah et al. "Deep clustering with a dynamic autoencoder: From reconstruction towards centroids construction". In: *Neural Networks* 130 (2020), pp. 206–228.

[25] D. A. Reynolds. "Gaussian Mixture Models". In: *Encyclopedia of Biometrics*. Vol. 741. Springer, 2009, pp. 659–663.

[26] Janik Schnellbach and Marton Kajo. "Clustering with deep neural networks–an overview of recent methods". In: *Network* 39.2 (2020), pp. 39–43.

[27] Uri Shaham and Roy R Lederman. "Learning by coincidence: Siamese networks and common variable learning". In: *Pattern Recognition* 74 (2018), pp. 52–63.

[28] Uri Shaham et al. "Spectralnet: Spectral clustering using deep neural networks". In: *arXiv preprint arXiv:1801.01587* (2018).

[29] Casper Kaae Sønderby et al. "Ladder variational autoencoders". In: *Advances in neural information processing systems* 29 (2016).

[30] Chunfeng Song et al. "Auto-encoder based data clustering". In: *Iberoamerican congress on pattern recognition*. Springer. 2013, pp. 117–124.

[31] Kai Tian, Shuigeng Zhou, and Jihong Guan. "Deepcluster: A general clustering framework based on deep learning". In: *Joint European conference on machine learning and knowledge discovery in databases*. Springer. 2017, pp. 809–825.

[32] Jakub Tomczak and Max Welling. "VAE with a VampPrior". In: *International conference on artificial intelligence and statistics*. PMLR. 2018, pp. 1214–1223.

[33] Tung-Long Vuong et al. "Vector quantized wasserstein auto-encoder". In: *arXiv preprint arXiv:2302.05917* (2023).

[34] Junyuan Xie, Ross Girshick, and Ali Farhadi. "Unsupervised deep embedding for clustering analysis". In: *International conference on machine learning*. PMLR. 2016, pp. 478–487.

[35] Wei Xu, Xin Liu, and Yihong Gong. "Document clustering based on non-negative matrix factorization". In: *Proceedings of the 26th annual international ACM SIGIR conference on Research and development in informaion retrieval*. 2003, pp. 267–273.

[36] Bo Yang et al. "Towards k-means-friendly spaces: Simultaneous deep learning and clustering". In: *international conference on machine learning*. PMLR. 2017, pp. 3861–3870.

[37] Zhongyuan Zhang et al. "Binary matrix factorization with applications". In: *Seventh IEEE international conference on data mining (ICDM 2007)*. IEEE. 2007, pp. 391–400.

[38] Sheng Zhou et al. "A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions". In: *ACM Computing Surveys* 57.3 (2024), pp. 1–38.

A Symmetric Non-negative Matrix Factorization

Ding and He [3] introduced the symmetric NMF of $W \approx HH^T$, where the pairwise similarity matrix $W = XX^T$ is the standard inner-product linear Kernel matrix. It can be extended to any other kernels. This is done using a non-linear transformation to the higher dimensional space

$$\mathbf{x}_i \to \phi(\mathbf{x}_i) \tag{78}$$

Based on this mapping, Eq. (6) can be written as

$$\mathcal{L}_\phi = \mathrm{S}' - \sum_{k=1}^{K} \frac{1}{n_k} \sum_{i \in C_k} \sum_{j \in C_k} \phi(x_i)^\top \phi(x_j), \tag{79}$$

where $\mathrm{S}' = \sum_i \|\phi(\mathbf{x}_i)\|^2$ is a constant. Let the Kernel matrix $W_{ij} = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$. Using the same method as in the previous section, by substituting indicator matrix H, we obtain

$$\mathcal{L}_\phi = -\sum_{k=1}^{K} \frac{1}{n_k} \sum_{i,j \in C_k} w_{ij} = -\mathrm{Tr}(H^T W H). \tag{80}$$

Symmetric NMF has the effects of kernel k-means clustering and Laplacian-based spectral clustering. The objective function of symmetric NMF can be written as

$$\min_{H \geq 0} \mathcal{L}_{\mathrm{NMF3}} = \|W - HH^T\|^2, \quad \text{s.t. } H^T H = I. \tag{81}$$

Relaxing this orthogonality constraint, we can easily prove the equivalence between symmetric NMF and k-means clustering.

Theorem 2. *Symmetric NMF of $W \approx HH^T$ is equivalent to kernel k-means clustering with the the strict orthogonality relation relaxed.*

Proof. The minimization of Eq. (80) can be written as

$$\begin{aligned} H &= \arg \min_{H^T H = I, H \geq 0} -2\mathrm{Tr}(H^T W H) \\ &= \arg \min_{H^T H = I, H \geq 0} -2\mathrm{Tr}(H^T W H) + \|H^T H\|^2 \\ &= \arg \min_{H^T H = I, H \geq 0} (\|W\|^2 - 2\mathrm{Tr}(H^T W H) + \|H^T H\|^2) \\ &= \arg \min_{H^T H = I, H \geq 0} \|W - HH^T\|^2. \end{aligned} \tag{82}$$

Which means Eq. (80) is is equivalent to Eq. (81). Relaxing the orthogonality constraint $H^T H = I$ completes the proof. □

When the orthogonality constraint in symmetric NMF is relaxed, the optimization objective still maintains a degree of near-orthogonality.

Theorem 3. *Symmetric NMF of $W \approx HH^T$ still remains the near-orthogonality of $H^T H = I$.*

Proof. Eq. (81) is equivalent to $\min_{H>0} \|H^T H\|^2$. We expand it as follows

$$\|H^T H\|^2 = \sum_{\ell k} (H^T H)^2_{\ell k} = \sum_{\ell \neq k} (\mathbf{h}_\ell^T \mathbf{h}_k)^2 + \sum_k (\mathbf{h}_k^T \mathbf{h}_k)^2. \tag{83}$$

The minimization of the first term ensures the orthogonality between $\mathbf{h}_\ell$ and $\mathbf{h}_k$ which means $\mathbf{h}_\ell \mathbf{h}_k \approx 0$. The minimization of the second term is equivalent to

$$\min \sum_{i=1}^{k} \|\mathbf{h}_i\|^4 \tag{84}$$

In Theorem 2, the optimization objective of Symmetric NMF has been proven to be equivalent to $\min -\mathrm{Tr}(H^T W H)$, which means H can not be zero. Simultaneously, given $W \approx HH^T$, we can derive

$$\sum_{ij} w_{ij} \approx \sum_{ij} (HH^T)_{ij} = \sum_{kij} h_{ik} h_{jk} = \sum_k |\mathbf{h}_k|^2, \tag{85}$$

where $|\mathbf{h}| = \sum_i |h_i| = \sum_i h_i$ is the L1 norm of vector $\mathbf{h}$, which means $\|\mathbf{h}\| \geq 0$. Consequently, the optimization of $\min_{H>0} \|H^T H\|^2$ suggests that H retains near-orthogonality, which can be described as follows

$$\mathbf{h}_\ell^T \mathbf{h}_k \approx \begin{cases} 0 & \text{if } \ell \neq k, \\ \|\mathbf{h}_k\|^2 & \text{if } \ell = k. \end{cases} \tag{86}$$

□

B Optimization Methods of Non-negative Matrix Factorization

Although the objective function of NMF defined in Eq. (9), is convex with respect to each variable individually, it becomes a non-convex problem when considering both variables together. Therefore, it is difficult to find the global minima. However, there are various techniques that can find the local minima. The most common methods is the multiplicative update rules and Alternating Least Squares(ALS).

Lee and Seung [14] proposed the multiplicative update rules, which is the most classic algorithm for optimization of NMF. The core idea of this method is to alternately update two factor matrices through an element-wise multiplicative procedure. At each iteration, each element in matrices Y and C is multiplied by a selected scaling factor, ensuring that the objective function decreases monotonically.

For the Euclidean distance used in Eq. (9), the multiplicative update rules are derived as follows

$$C \leftarrow C \circ \frac{Y^T X}{Y^T Y C} \tag{87}$$

$$Y \leftarrow Y \circ \frac{X C^T}{Y C C^T} \tag{88}$$

where $\circ$ is the Hadamard product, i.e., element-wise multiplication. In the iterative procedure, matrix C is updated by multiplying element-wise with a factor $\frac{Y^T X}{Y^T Y C}$ which is computed based on the current parameter values. Then, in the same way, matrix Y is updated by multiplying element-wise with the factor $\frac{XC^T}{YCC^T}$. This process is repeated until convergence. Multiplicative update rules is easy to implement which only involves matrix multiplications and element-wise operations. Moreover, as long as both Y and C are initialized with non-negative values, the non-negativity will be maintained during each iteration.
In real-world datasets, the data may contain negative values, thus the centroid matrix C may not be non-negative. We cam relax the non-negative constraint $C \geq 0$ in Eq. (9). Under this condition, the optimization of the loss function becomes a Semi-NMF optimization problem [4]. The multiplicative updating rules for Semi-NMF are as follows

$$C \leftarrow XY^\top (YY^\top)^{-1} \tag{89}$$

$$Y_{ik} \leftarrow Y_{ik} \sqrt{\frac{[(X^\top C)^+]_{ik} + [Y(C^\top C)^-]_{ik}}{[(X^\top C)^-]_{ik} + [Y(C^\top C)^+]_{ik}}} \tag{90}$$

Another widely used optimization method is ALS [20]. Its core idea is to decompose the NMF optimization problem into two subproblems with non-negativity constraints, which are solve iteratively. Specifically, the algorithm alternately fixes one matrix and optimizes the other, progressively converging toward the optimal solution. During the optimization process, give data matrix X, we firstly initialize two non-negative matrices $Y \geq 0$ and $C \geq 0$. Then, in each iteration, we fix $Y^{(t)}$, optimize C by solving

$$\min_{C \geq 0} \left\| X - Y^{(t)} C \right\|^2 \tag{91}$$

Then fix $C^{(t+1)}$, optimize Y by solving

$$\min_{Y \geq 0} \left\| X - Y C^{(t+1)} \right\|^2 \tag{92}$$

The above steps are repeated iteratively until the loss function converges.

C Further Experimental Results

The Siamese network requires additional training steps; therefore, in the main text we only compare SpectralNet that uses the Euclidean distance. The Siamese network is trained on a set of positive and negative sample pairs, where the Euclidean distance is used to label the pairs. In experiments, positive pairs are constructed from each point's nearest neighbors, while negative pairs are formed from points that are far apart. The Siamese network maps each data point x_i to an embedding $z_i = G_{Siamese}(x_i)$ in some space. The network typically minimizes the following contrastive loss

$$L_{\text{Siamese}}\big(\theta_{\text{Siamese}}; x_i, x_j\big) = \begin{cases} \|\mathbf{z}_i - \mathbf{z}_j\|^2, & (x_i, x_j) \text{ positive}, \\ \max\big(c - \|\mathbf{z}_i - \mathbf{z}_j\|, 0\big)^2, & (x_i, x_j) \text{ negative}. \end{cases} \tag{93}$$

Table 4: Clustering performance on **Fashion MNIST** (ACC, NMI, ARI, F1, and Purity) compared across different methods *(best in each column shown in **bold**.*

Method	**ACC**	**NMI**	**ARI**	**F1**	**Purity**
k-means	0.5107	0.4885	0.3437	0.4952	0.5470
NMF	0.4569	0.4757	0.3109	0.4168	0.4773
NMF(k-means++ Init)	0.4954	0.5372	0.3613	0.4593	0.5357
AE+k-means	0.5749	0.5406	0.3914	0.5734	0.5917
AE+NMF	0.5621	0.5859	0.3974	0.5334	0.5997
VAE+k-means	0.6055	0.5898	0.4129	0.6068	0.6194
VAE+NMF	0.5826	0.5638	0.3644	0.5244	0.5982
GMM	0.4963	0.5406	0.3531	0.4521	0.5296
DEC	0.6065	0.6130	0.4554	0.5506	0.6224
DC+k-means	0.5764	0.6045	0.4434	0.5410	0.6029
DC+GMM	0.6045	0.6211	0.4659	0.5591	0.6308
SpectralNet(Euclidean)	0.5765	0.6642	0.4681	0.5461	0.6605
DCN	0.5890	0.5993	0.4416	0.4996	0.5755
VAE-NMF (Fully Connected)	0.6375	0.6037	0.4808	0.6086	0.6443
VAE-NMF (Convolutional)	0.6504	0.6112	0.5003	0.6163	0.6619

where c is the margin (typically set to 1). After the Siamese network is trained, it is used to build the affinity matrix W for SpectralNet, which markedly improves clustering quality. The results are presented in Table 5.

Table 5: Clustering performance of **SpectralNet (Siamese)** across different datasets.

Dataset	**ACC**	**NMI**	**ARI**	**F1**	**Purity**
MNIST	0.9722	0.9271	0.9400	0.9722	0.9722
Reuters-10K	0.6632	0.4330	0.3243	0.6445	0.7139
Fashion MNIST	0.6219	0.6965	0.5084	0.6059	0.7086